

РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ И РАЗРАБОТЧИКА

Raw Image Process

Приложение для Windows, которое открывает RAW-файлы камер, измеряет шум сенсора, подавляет его прямо в мозаике Байера и восстанавливает цвет тремя методами демозаика: на процессоре или на видеокарте NVIDIA через CUDA.

СИСТЕМА

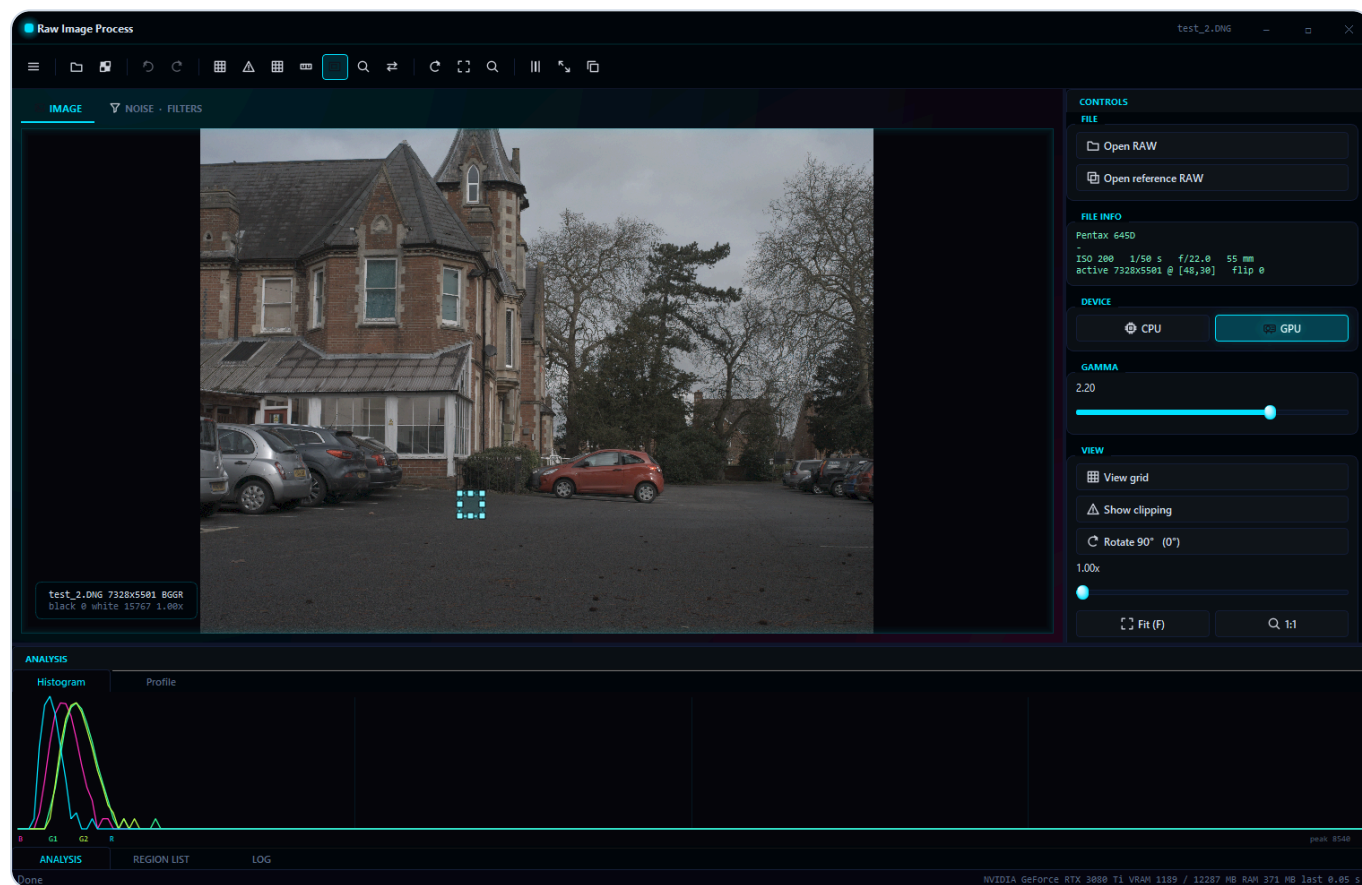
Windows 10 1809+ или 11, x64

ВЫЧИСЛЕНИЯ

CPU на всех ядрах, CUDA – если есть

ОСНОВА

C++17 · Qt 6.7 · LibRaw · CUDA 13



Главное окно: кадр Pentax 645D после демозаика Malvar, внизу гистограмма выделенной области

Содержание

Обзор

Возможности
Установка
Быстрый старт

Документация
Структура проекта
Сборка

Интерфейс

Окно целиком
Главное меню
Панель инструментов
Работа с изображением
Панель CONTROLS

Нижние панели
Вкладка NOISE · FILTERS
Строка состояния
Горячие клавиши
Типичные сценарии

Шум

Откуда берётся шум
Единицы и уровни
Почему шум считается по каналам
Статистика области
Полезные биты
Как правильно мерить шум

Гистограмма и профиль
Гистограммы до и после
Двухкадровый метод
Темновой кадр
Optical black
Фильтры шумоподавления

Цвет

Мозаика Байера
Подготовка значений
Binning 2×2
Bilinear
Malvar-He-Cutler

CPU и GPU
Гамма
Сравнение методов
Форматы файлов

Справочник функций

Как собрать свою программу
Готовые примеры
Типы данных
im_read.hpp — загрузка, демозаик, сохранение
image_stats.hpp — статистика и шум
denoise.hpp — шумоподавление

process.hpp — выбор устройства и метода
gpu_process.hpp — CUDA
im_process.cuh — ядра CUDA
main.cpp — консольная проверка
gui.cpp — приложение

Сборка и GitHub

Что установить для сборки
Сборка для разработки (Debug)
Release, папка программы и установщик

GitHub
Установка на другой компьютер
Частые проблемы

Обзор

Возможности

- **Чтение RAW через LibRaw:** активная область сенсора, уровни black и white, баланс белого камеры, EXIF, зона optical black.
- **Статистика любой области** отдельно по каналам R, G1, G2, B: среднее, σ , минимум, максимум, гистограмма и профиль.
- **Анализ шума:** гистограммы «до / после», SNR в dB, двухкадровый метод (временной шум и фиксированный узор), вычитание темного кадра, измерение уровня чёрного.
- **Шумоподавление в мозаике:** Gaussian, Median 3×3 / 5×5, Bilateral. Каждый CFA-канал фильтруется отдельно, цвета не смешиваются, всё отменяется через **Ctrl + Z**.
- **Демозаик:** Binning 2×2, Bilinear, Malvar-He-Cutler на CPU (все ядра) и на GPU (CUDA).
- **Экспорт:** RAW в **.bin** и 16-битный TIFF, результат в PNG и линейный 16-битный TIFF, статистика в CSV, пресеты в JSON.
- **Работает без видеокарты:** если CUDA-устройства нет, всё считается на процессоре.

Установка

1. Скачай **RawImageProcess-1.0-Setup.exe** со страницы **Releases** этого репозитория.
2. Запусти установщик. Если Windows покажет «Windows защитила ваш компьютер», нажми **Подробнее** → **Выполнить в любом случае**: у установщика нет платной цифровой подписи.
3. Открой **Raw Image Process** из меню Пуск.

Требования: Windows 10 версии 1809 или новее либо Windows 11, 64-бит; от 8 ГБ ОЗУ. Видеокарта NVIDIA не обязательна.

Быстрый старт

1. **Ctrl + 0** — открыть RAW. Файл можно просто перетащить в окно.
2. Протяни мышью прямоугольник по ровному однотонному участку: справа появится статистика, внизу гистограмма.
3. **Ctrl + 2** — вкладка шума: σ , SNR и гистограммы по каналам.
4. Выбери фильтр и нажми **Apply**, затем сравни кривые «до» и «после». **Ctrl + Z** отменяет фильтр.
5. **Ctrl + 1**, метод **Malvar**, **Ctrl + D** — цветная картинка. **Ctrl + S** сохраняет PNG.

Документация

РАЗДЕЛ	О ЧЁМ
Интерфейс	каждое меню, кнопка, панель и горячая клавиша; типичные сценарии работы
Шум	откуда берётся шум RAW, как программа его измеряет, как устроены фильтры
Восстановление цвета	мозаика Байера, три метода демозаика, гамма, форматы файлов
Справочник функций	все функции библиотеки с параметрами и примерами кода
Сборка и GitHub	сборка из исходников, установщик, публикация и установка на другой ПК

Структура проекта

ФАЙЛ	ЧТО ВНУТРИ
<code>gui.cpp</code>	приложение с окном на Qt 6
<code>im_read.hpp</code>	загрузка RAW, демозаик на CPU, сохранение файлов, гамма
<code>image_stats.hpp</code>	статистика, гистограммы, профиль, двухкадровый шум, темновой кадр
<code>denoise.hpp</code>	фильтры шумоподавления по CFA-каналам
<code>process.hpp</code>	выбор устройства (CPU/GPU) и метода демозаика
<code>gpu_process.hpp</code>	CUDA: память, запуск ядер, информация об устройстве
<code>im_process.cu</code> , <code>im_process.cuh</code>	CUDA-ядра демозаика и гаммы
<code>main.cpp</code>	консольная программа для быстрой проверки файла
<code>installer/</code>	скрипт сборки установщика и сценарий Inno Setup
<code>docs/</code>	документация, скриншоты и примеры кода

Сборка

Полная инструкция — в [docs/build-and-github.md](#). Готовую программу и установщик собирает одна команда:

```
powershell -ExecutionPolicy Bypass -File installer\build_installer.ps1
```

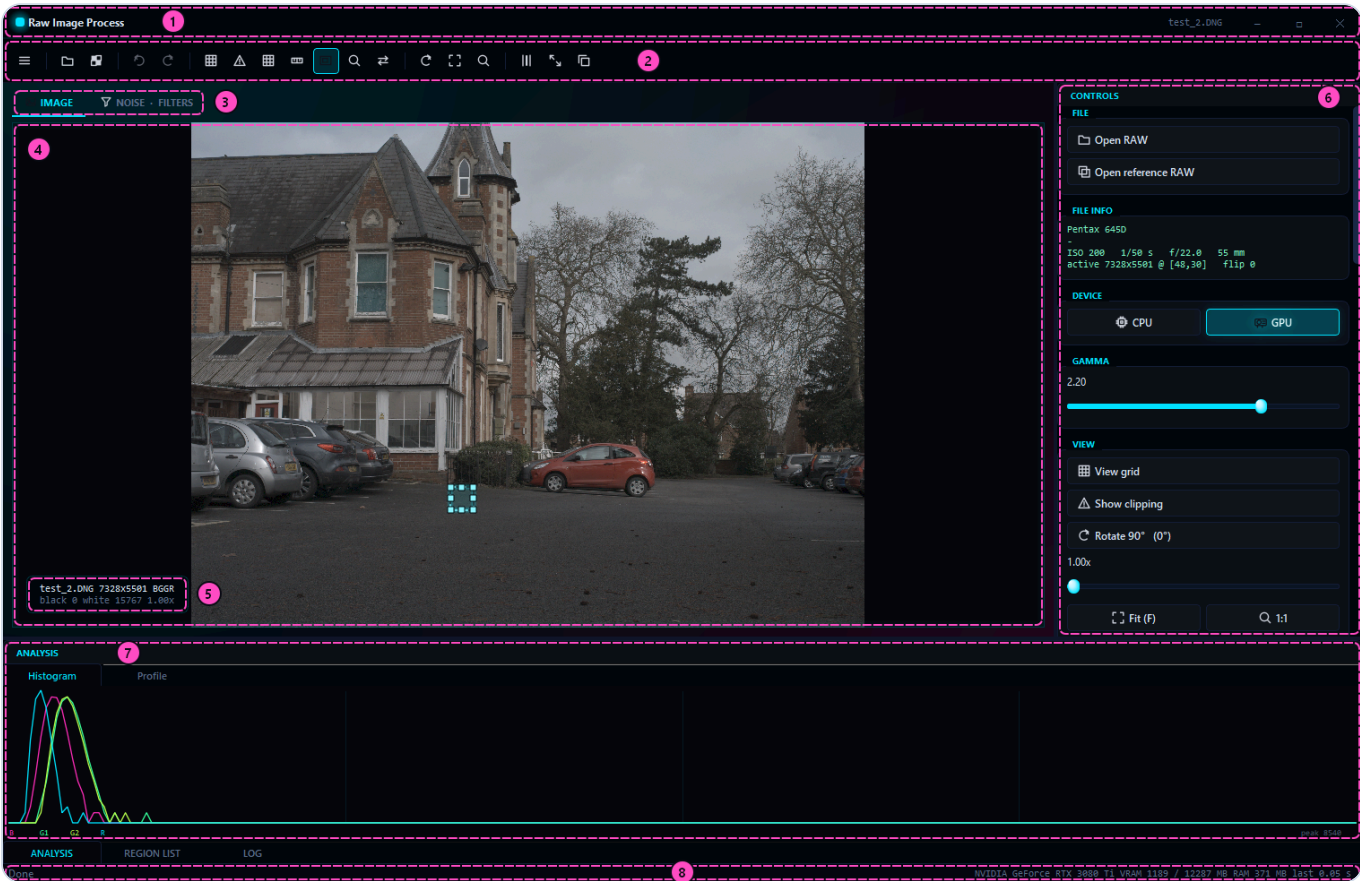
POWERSHELL

Скрипт собирает Release, складывает программу в `dist\RawImageProcess`, делает portable-zip и, если установлен Inno Setup 6, `dist\RawImageProcess-1.0-Setup.exe`.

Интерфейс

Здесь описана каждая часть окна: что делает кнопка, пункт меню или горячая клавиша и когда это нужно. Скриншоты сняты с кадра Pentax 645D (test_2.DNG , 7328×5501).

Окно целиком



Главное окно, цифрами отмечены основные части

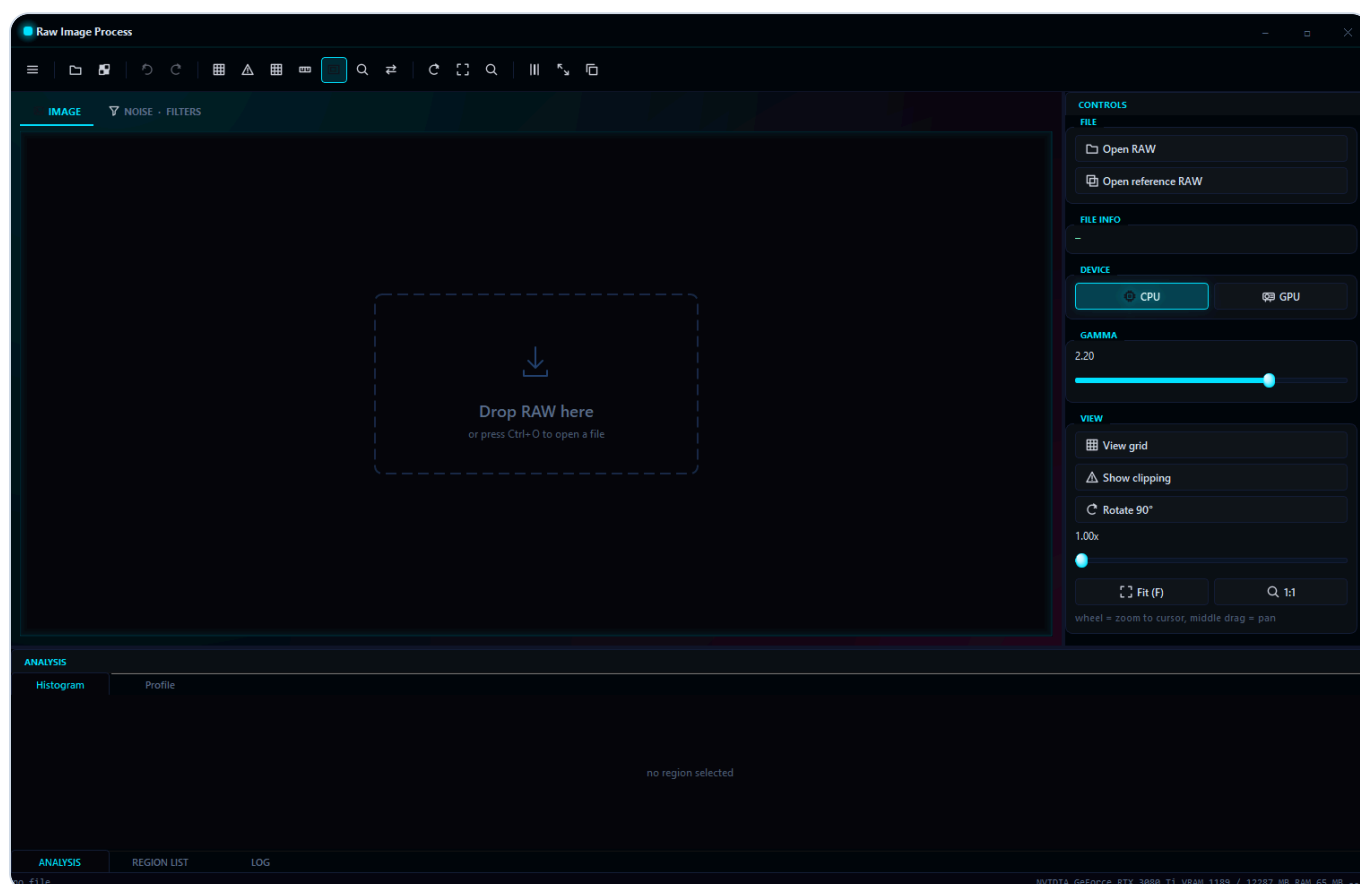
№	ЧАСТЬ ОКНА	ЧТО ДЕЛАЕТ
1	Заголовок	название программы и открытого файла; кнопки «свернуть», «развернуть», «заккрыть». За эту полосу окно перетаскивается
2	Панель инструментов	кнопка ≡ с главным меню и быстрые кнопки
3	Вкладки страниц	IMAGE – фото, NOISE · FILTERS – шум и фильтры. Если открыто несколько файлов, рядом появляются вкладки файлов
4	Область просмотра	RAW или цветной результат, выделение областей, лупа, сетка, сравнение
5	HUD	файл, размер, узор CFA; под курсором – координаты, канал и значение; во время обработки – процент выполнения
6	Панель CONTROLS	кнопки обработки, анализа и экспорта

№	ЧАСТЬ ОКНА	ЧТО ДЕЛАЕТ
7	Нижние панели	ANALYSIS (гистограмма и профиль), REGION LIST, LOG
8	Строка состояния	данные файла, видеокарта, память, время последней операции, значение под курсором

Панели CONTROLS, ANALYSIS, REGION LIST и LOG можно перетаскивать за заголовок, складывать вкладками и откреплять в отдельные окна. Расположение запоминается при выходе. **Tab** прячет все панели сразу.

Пустое окно

Пока файл не открыт, в центре подсказка **Drop RAW here**. Перетащи файл из Проводника или нажми **Ctrl + O**.

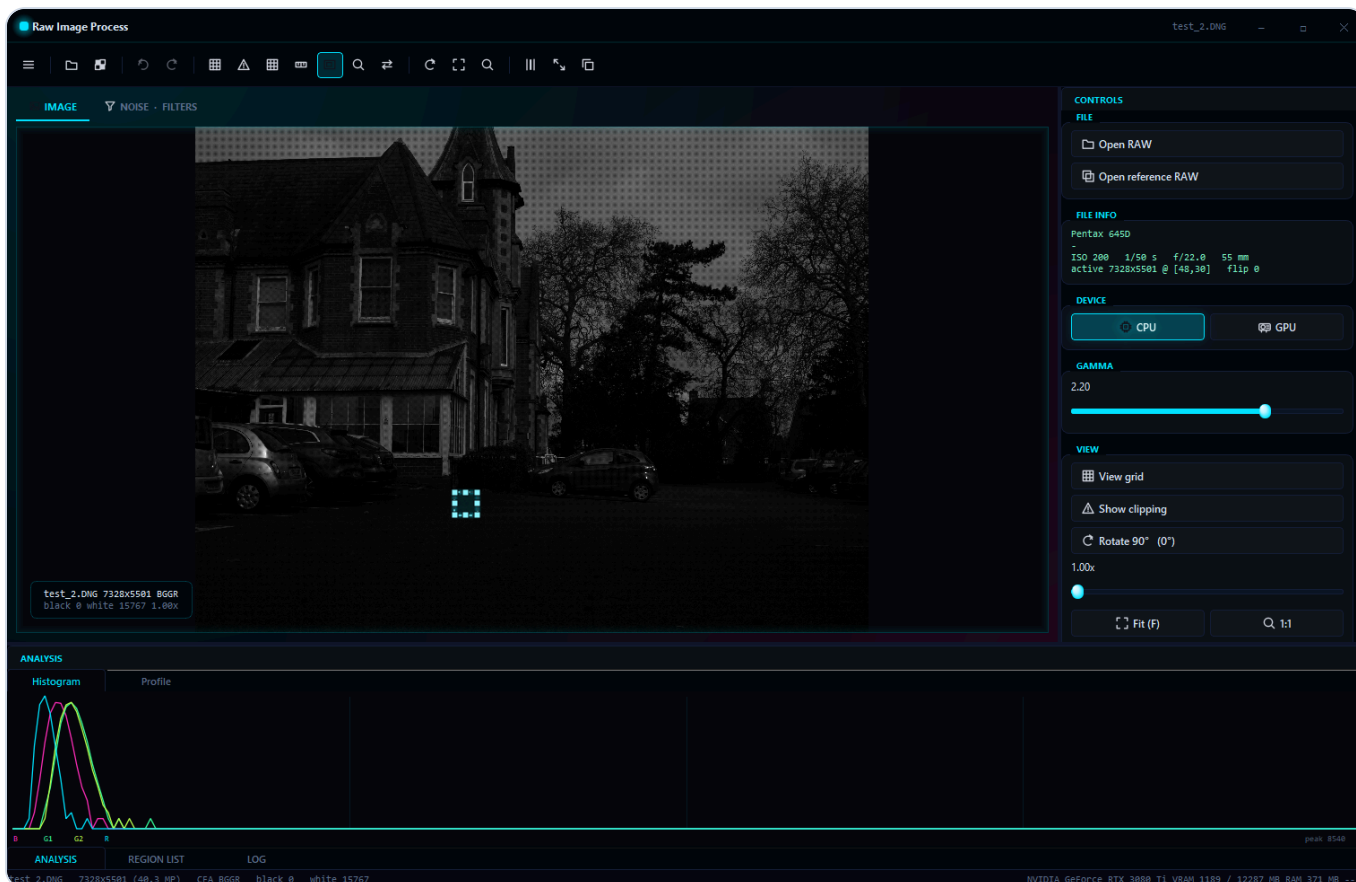


Окно без открытого файла

Программа открывает RAW-файлы, которые понимает LibRaw и у которых мозаика Байера: DNG, CR2, CR3, NEF, ARW, ORF, RW2 и другие. Не открываются файлы с сенсором X-Trans (часть камер Fujifilm) и «линейные» DNG, в которых цвет уже восстановлен камерой или телефоном: в них нет мозаики, которую нужно обрабатывать.

RAW до демозаика

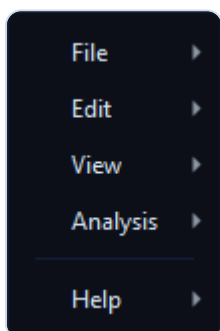
Сразу после открытия показаны исходные значения сенсора: серым, от `black level` (чёрное) до `white level` (белое). Мелкая «клетка» на картинке — это и есть мозаика Байера. Цвета здесь ещё нет, подробнее — в разделе [Восстановление цвета](#).



RAW после открытия файла, выделена ровная область асфальта

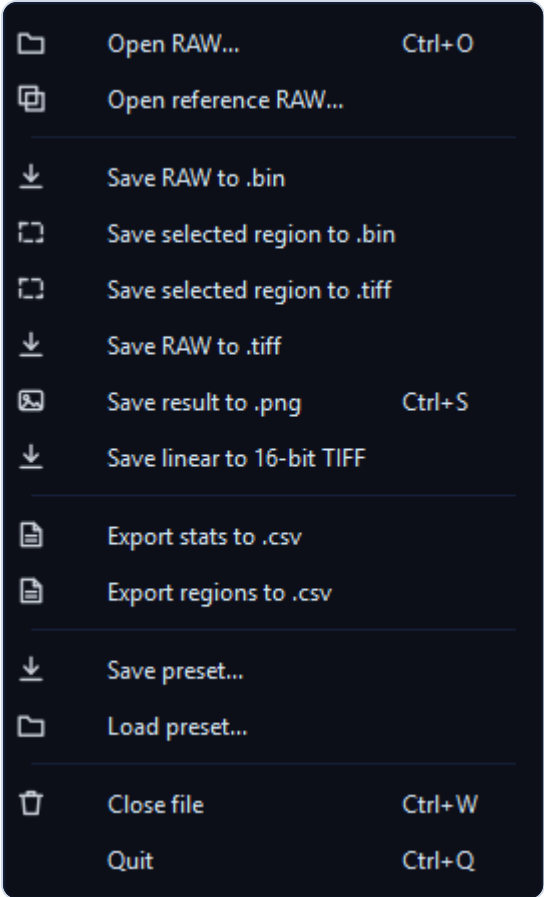
Главное меню

Кнопка  слева на панели инструментов.



Главное меню

File



Меню File

ПУНКТ	КЛАВИША	ЧТО ДЕЛАЕТ
Open RAW...	Ctrl + O	открыть RAW; каждый файл открывается в своей вкладке
Open reference RAW...		загрузить второй кадр того же размера — для двухкадрового шума и вычитания темного кадра
Save RAW to .bin		сохранить значения сенсора как есть, 16 бит без заголовка
Save selected region to .bin		то же, но только выделенная область — так файл читается в Mathcad и MATLAB без зависаний
Save selected region to .tiff		выделенная область как 16-битный серый TIFF без обработки
Save RAW to .tiff		сохранить мозаику как 16-битный серый TIFF
Save result to .png	Ctrl + S	сохранить цветную картинку после демозаика и гаммы
Save linear to 16-bit TIFF		цвет без гаммы, 16 бит на канал, половинный размер
Export stats to .csv		статистика выделенной области в таблицу
Export regions to .csv		статистика всех областей из REGION LIST

ПУНКТ	КЛАВИША	ЧТО ДЕЛАЕТ
Save preset... / Load preset...		сохранить или загрузить настройки и области в JSON
Close file	Ctrl + W	заккрыть текущий файл
Quit	Ctrl + Q	ВЫЙТИ

Форматы файлов подробно разобраны в разделе [Восстановление цвета](#).

Edit



Меню Edit

ПУНКТ	КЛАВИША	ЧТО ДЕЛАЕТ
Undo	Ctrl + Z	отменить последнее изменение
Redo	Ctrl + Y	вернуть отменённое

Отменяются фильтры шума, вычитание темнового кадра, смена black level, возврат к оригиналу, добавление и удаление областей. Хранятся последние 12 шагов. Изменения вида (зум, сетка, поворот) в историю не попадают.

View



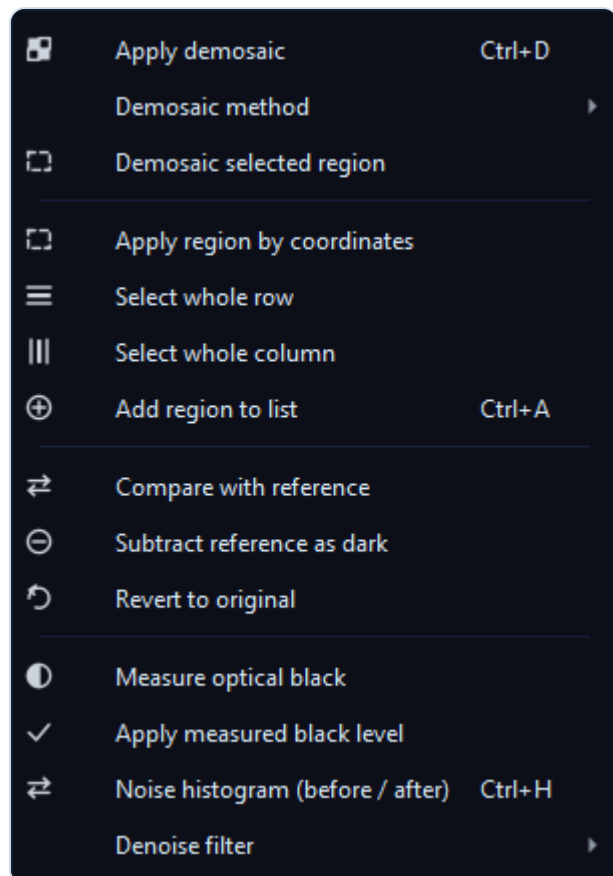
Меню View

Всё в этом меню меняет только то, как видно изображение. Данные не трогаются.

ПУНКТ	КЛАВИША	ЧТО ДЕЛАЕТ
Pixel grid	G	сетка между пикселями; видна, когда один пиксель занимает на экране от 4 точек
Clipping	C	красным отмечает пересвет (значение $\geq$ white level), синим — провал в чёрное ($\leq$ black level)
Rule of thirds	T	линии «правила третей»
Rulers	U	линейки с координатами пикселей сверху и слева
Minimap	M	уменьшенный кадр в углу с рамкой видимой части; появляется при увеличении, включена по умолчанию
Loupe	L	лупа $\times 8$ у курсора с координатами, каналом и значением
Compare before / after	B	слева RAW, справа результат демозаика; разделитель тянется мышью
Rotate 90	R	повернуть вид на 90°; координаты областей остаются в системе сенсора
Fit to window	F	весь кадр в окне
Zoom 1:1	1	один пиксель результата на одну точку экрана
Full screen	F11	полный экран
Hide panels	Tab	спрятать панели, панель инструментов и строку состояния

ПУНКТ	КЛАВИША	ЧТО ДЕЛАЕТ
Copy view to clipboard	Ctrl + Shift + C	скопировать видимое фото в буфер обмена

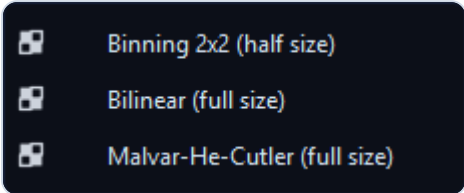
Analysis



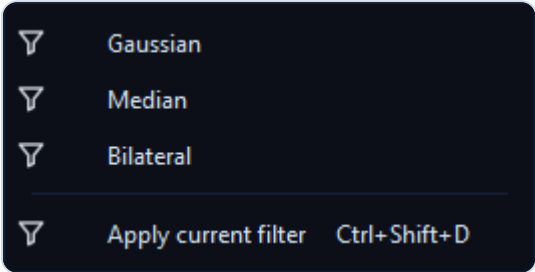
Меню Analysis

ПУНКТ	КЛАВИША	ЧТО ДЕЛАЕТ
Apply demosaic	Ctrl + D	собрать цветную картинку выбранным методом на выбранном устройстве
Demosaic method		подменю: Binning 2x2, Bilinear, Malvar-He-Cutler
Demosaic selected region		демозаик только выделенной области — быстро на больших файлах
Apply region by coordinates		выделить область по полям X, Y, W, H панели CONTROLS
Select whole row / Select whole column		выделить строку номер Y или столбец номер X целиком
Add region to list	Ctrl + A	добавить текущее выделение в REGION LIST
Compare with reference		двухкадровый шум: временной, фиксированный узор, полный
Subtract reference as dark		вычесть темновой кадр
Revert to original		вернуть кадр в состояние при открытии

ПУНКТ	КЛАВИША	ЧТО ДЕЛАЕТ
Measure optical black		измерить закрытые от света пиксели по краю сенсора
Apply measured black level		заменить black level файла на измеренный
Noise histogram (before / after)	Ctrl + H	открыть вкладку NOISE · FILTERS
Denoise filter		подменю: Gaussian, Median, Bilateral и Apply current filter (Ctrl + Shift + D)



Подменю Demosaic method



Подменю Denoise filter

Help

About показывает версию программы, дату сборки, версии Qt, CUDA и LibRaw, название видеокарты.

Панель инструментов



Панель инструментов

Слева направо:

КНОПКА	КЛАВИША	ДЕЙСТВИЕ
☰		главное меню
Открыть	Ctrl + O	Open RAW
Мозаика	Ctrl + D	Apply demosaic
Стрелка назад / вперёд	Ctrl + Z / Ctrl + Y	Undo / Redo
Сетка	G	Pixel grid

КНОПКА	КЛАВИША	ДЕЙСТВИЕ
Треугольник	C	Clipping
Сетка 3×3	T	Rule of thirds
Линейка	U	Rulers
Карта	M	Minimap
Лупа	L	Loupe
Две стрелки	B	Compare before / after
Круговая стрелка	R	Rotate 90
Рамка	F	Fit to window
Лупа 1:1	1	Zoom 1:1
Столбцы		Select whole column
Стрелки наружу	F11	Full screen
Два листа	Ctrl + Shift + C	Copy view to clipboard

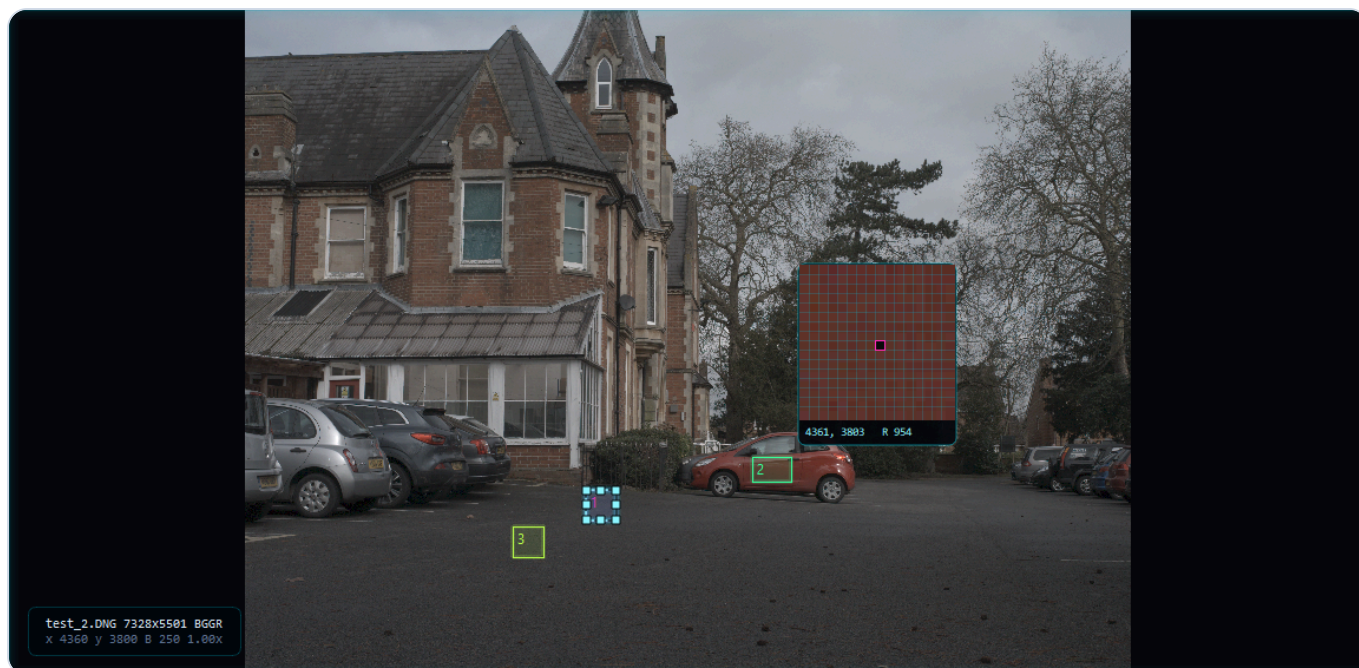
Если навести мышь на кнопку, появится её название.

Работа с изображением

ДЕЙСТВИЕ	КАК
Увеличить или уменьшить	колесо мыши; увеличивается точка под курсором
Сдвинуть кадр	зажать колесо мыши и тянуть
Выделить область	левой кнопкой протянуть прямоугольник
Изменить размер области	тянуть за маркеры на углах и сторонах рамки
Сдвинуть область на 1 пиксель	стрелки; сначала кликни по фото
Сдвинуть область на 10 пикселей	Shift + стрелки
Открыть файл	перетащить его в окно

Лупа

Лупа (**L**) увеличивает участок под курсором в 8 раз. Подпись внизу — координаты пикселя сенсора, его канал и значение в ADU. Те же данные показывает HUD.



Лупа над красной машиной: пиксель 4361, 3803 канала R, значение 954

Сетка пикселей



Сетка пикселей при сильном увеличении

Линейки и миникарта

При увеличении в углу появляется миникарта: рамка показывает, какая часть кадра сейчас на экране.



Линейки с координатами и миникарта

Клиппинг



Режим Clipping: пересвеченные пиксели красные, провалы в чёрное синие

Сравнение RAW и результата



Compare before / after: слева RAW, справа результат Malvar

Панель CONTROLS

Все группы панели сверху вниз. Если панель не помещается по высоте, её можно прокрутить колесом мыши.

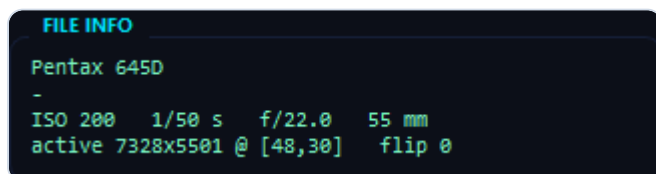
FILE



Группа FILE

Open RAW открывает файл, **Open reference RAW** — второй кадр того же размера для двухкадрового шума и темного кадра.

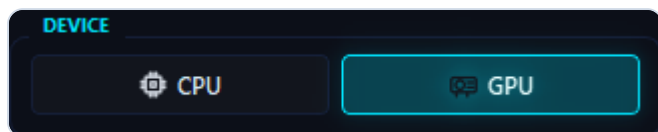
FILE INFO



Группа FILE INFO

Камера, объектив, ISO, выдержка, диафрагма, фокусное расстояние. Строка `active 7328x5501 @ [48, 30]` — размер активной (светочувствительной) области и её смещение от края полного кадра сенсора. `flip` — поворот, записанный камерой; программа сразу применяет его к виду.

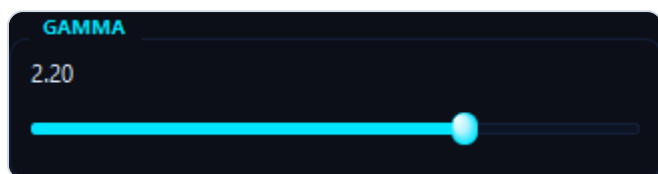
DEVICE



Группа DEVICE

На чём считать демозаик и гамму: **CPU** или **GPU**. Если видеокарты NVIDIA с CUDA нет, кнопка GPU серая и всё считается на процессоре. Фильтры шума и статистика всегда работают на CPU.

GAMMA



Группа GAMMA

Гамма цветной картинки от 0.10 до 3.00, стандарт — 2.20. Меняет только вид результата демозаика и сохраняемый PNG; RAW-данные и статистику не трогает.

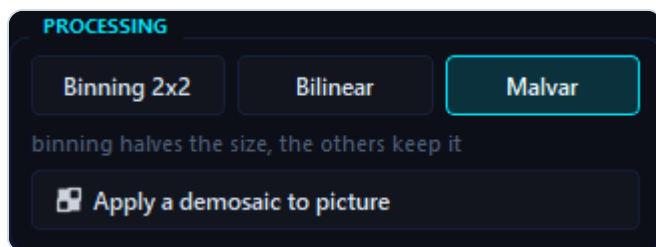
VIEW



Группа VIEW

Сетка пикселей, клиппинг, поворот, ползунок зума, кнопки **Fit** и **1:1**. Подсказка внизу напоминает: колесо — зум, средняя кнопка — перемещение.

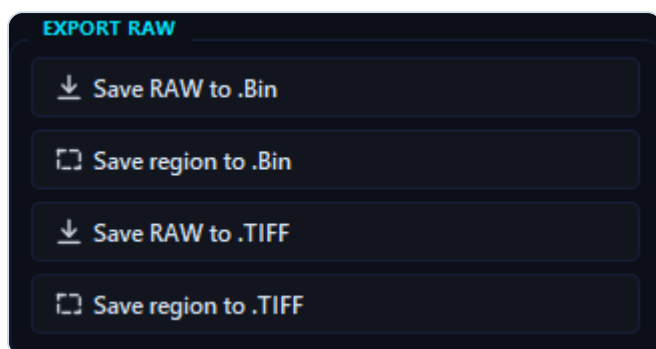
PROCESSING



Группа PROCESSING

Метод демозаика (**Binning 2x2**, **Bilinear**, **Malvar**) и кнопка **Apply a demosaic to picture**. Во время работы появляются полоса прогресса и кнопка **Cancel**.

EXPORT RAW



Группа EXPORT RAW

Save RAW to .Bin сохраняет весь кадр числами, **Save region to .Bin** — только выделенную область, **Save RAW to .TIFF** — мозаику всего кадра как 16-битный серый TIFF, **Save region to .TIFF** — мозаику выделенной области. Ни один из этих файлов не проходит через демозаик, гамму или фильтры: в них исходные значения сенсора.

Каждое сохранение уходит в **свою папку**, названную по имени файла, — программа создаёт её сама, ничего заранее готовить не нужно. Внутри лежит сам файл и текстовый блокнот с размерами:

- `имя_mathcad.txt` рядом с `.bin` — первой строкой готовая команда `READBIN(...)` для Mathcad, дальше ширина, высота, размер в байтах, уровни black и white, узор CFA и камера;
- `имя_info.txt` рядом с `.tiff` — то же самое без строки `READBIN`, размеры у TIFF есть в заголовке.

Без размеров двоичный файл нельзя прочитать правильно. Подробности — в разделе [Восстановление цвета](#).

PROCESSING RESULT

PROCESSING RESULT

 Save to .png

 Save linear to 16-bit TIFF

Группа PROCESSING RESULT

Сохранить цветной результат в PNG или линейный 16-битный TIFF.

REGION BY MOUSE

REGION BY MOUSE

[2816, 3968] 256x256 px

 Demosaic this region

Группа REGION BY MOUSE

Координаты области, выделенной мышью, и кнопка **Demosaic this region**.

REGION BY COORDINATES

REGION BY COORDINATES

X: 2816

Y: 3968

W: 256

H: 256

 Apply region(coordinates)

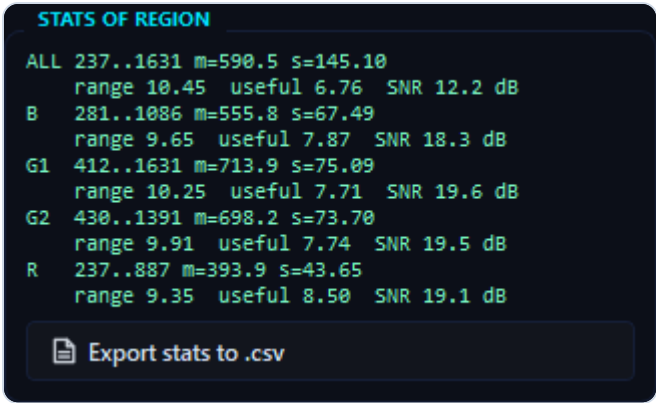
 Whole row

 Whole column

Группа REGION BY COORDINATES

Поля **X**, **Y**, **W**, **H** и кнопка **Apply region(coordinates)** — выделить область точно по числам; вид сразу переходит к ней. **Whole row** выделяет всю строку номер **Y**, **Whole column** — весь столбец номер **X**. При выделении мышью поля заполняются сами.

STATS OF REGION



Группа STATS OF REGION

Статистика выделенной области: по две строки на канал.

B

281..1086 m=555.8 s=67.49

range 9.65 useful 7.87 SNR 18.3 dB

ТЕКСТ

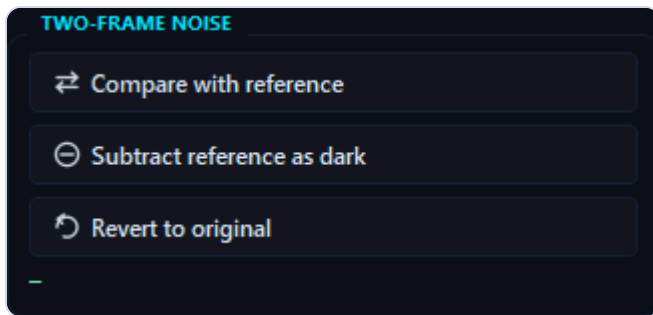
часть	значение
B	канал; первая строка ALL — все пиксели вместе
281..1086	минимум и максимум в ADU
m	среднее
s	стандартное отклонение σ ; на ровном участке это шум
range	$\log_2(\max - \min + 1)$: сколько бит занимает разброс значений в области
useful	$\log_2((\text{white} - \text{black}) / \sigma)$: сколько уровней различимо над шумом — полезные биты
SNR	$20 \cdot \log_{10}((\text{среднее} - \text{black}) / \sigma)$ — отношение сигнал/шум

Подробно про эти три числа — в разделе Шум.

Шум читают по строкам каналов, а не по ALL. На скриншоте у ALL $\sigma = 145.10$, а у каналов — от 43.65 до 75.09: при смешивании каналов с разными средними σ завышается.

Export stats to .csv сохраняет эти числа вместе с данными файла.

TWO-FRAME NOISE



Группа TWO-FRAME NOISE

Работает, когда загружен reference RAW:

- **Compare with reference** — шум по двум кадрам: `rd` (временной), `fpn` (фиксированный узор), `tot` (полный) для каждого канала;
- **Subtract reference as dark** — вычесть темновой кадр;
- **Revert to original** — вернуть кадр в состояние при открытии.

Как это считается — в разделе [Шум](#).

OPTICAL BLACK



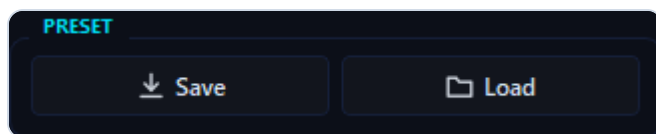
Группа OPTICAL BLACK после измерения на кадре Pentax 645D

Measure black from optical area измеряет закрытые от света пиксели по краю сенсора: среднее (уровень чёрного) и σ (шум считывания), всего и по каналам. Последняя строка — **динамический диапазон сенсора**: $\log_2((\text{white} - \text{black}) / \sigma \text{ шума считывания})$, в битах и в децибелах. **Apply as black level** записывает измеренное среднее в black level файла.

ВНИМАНИЕ

На скриншоте $\sigma = 335.86$ ADU при среднем 49.61. Такой огромный разброс значит, что полоса по краю этого файла не полностью тёмная, и применять это значение нельзя. У честной зоны optical black σ обычно единицы ADU. Если в файле такой зоны нет (например, у DNG с iPhone), программа так и напишет.

PRESET

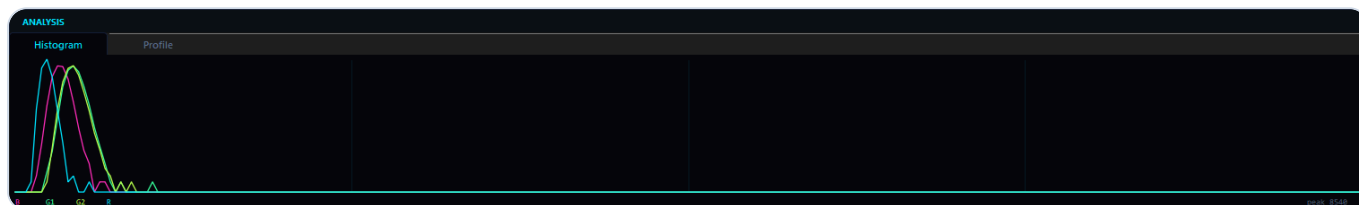


Группа PRESET

Save сохраняет в JSON гамму, устройство, сетку, клиппинг, поворот, зум, список областей и текущее выделение. **Load** возвращает всё обратно. Удобно, когда нужно промерить серию снимков по одним и тем же областям.

Нижние панели

ANALYSIS → Histogram



Гистограмма выделенной области: отдельная кривая для каждого канала

Распределение значений в выделенной области: 256 столбцов от 0 до white level, отдельная цветная кривая для каждого канала. Справа внизу — высота самого высокого столбца (`peak`).

ANALYSIS → Profile



Профиль вдоль строки через машины

Среднее значение по каждому столбцу области. Если область шириной в один пиксель, — по каждой строке. Удобно искать виньетирование, полосы и перепады освещения. На скриншоте выделена целая строка через машины.

REGION LIST

REGION LIST

#	X	Y	W	H	mean	sigma
1	2816	3968	256	256	590.5	145.10
2	4200	3700	320	200	657.9	398.84
3	2216	4268	256	256	584.0	157.30

⊕ Add

⊖ Remove

🗑 Clear

📄 Export all regions to .csv

Список сохранённых областей

Сохранённые области: номер, X, Y, W, H, среднее и σ .

- Клик по строке выделяет эту область и переносит к ней вид.
- Цвет номера совпадает с цветом рамки на фото.
- **Add** добавляет текущее выделение, **Remove** удаляет выбранную строку, **Clear** очищает список.
- **Export all regions to .csv** сохраняет статистику всех областей по каналам.

LOG

```
LOG
11:47:00 loaded test_2.DWG 7328x5501 active [48,30]
11:47:01 demosaic method: Halvor-Hu-Cutler
11:47:02 done: Demosaic (Halvor-Hu-Cutler)... (0.05 s)
11:47:08 optical black: 49.61 ADU, read noise 335.86 ADU
11:47:09 region 1 added: [2816, 3968] 256x256
11:47:09 region 2 added: [4200, 3700] 320x200
11:47:09 region 3 added: [2216, 4268] 256x256
```

Журнал действий

Журнал со временем: какой файл открыт, какие операции выполнены и сколько секунд они заняли, как изменился шум после фильтра.

Вкладка NOISE • FILTERS

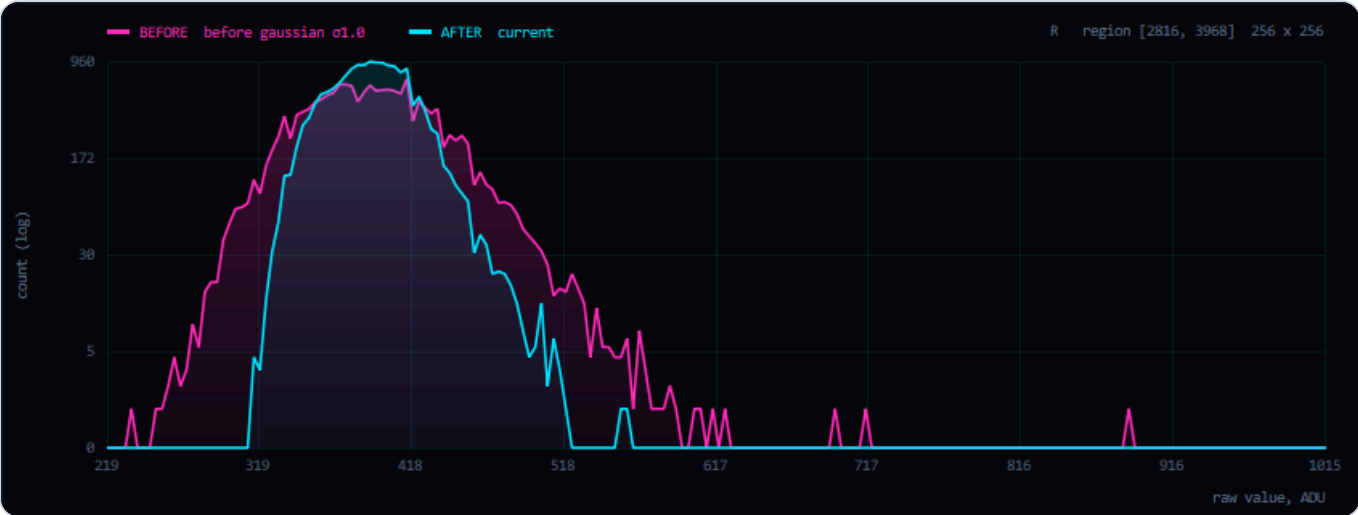
Ctrl + **2** или вкладка над изображением. Здесь сравниваются гистограммы шума до и после обработки и применяются фильтры.



Вкладка NOISE · FILTERS после фильтра Gaussian σ 1.0 на асфальте

ЭЛЕМЕНТ	ЧТО ДЕЛАЕТ
ALL / R / G1 / G2 / B	какой канал показывать; ALL — все четыре вместе
Noise	гистограмма отклонений от среднего канала — «форма шума»
Values	гистограмма самих значений
Log scale	логарифмическая вертикальная шкала, чтобы были видны редкие значения
Карточки	σ до, σ после, изменение в процентах, SNR после в dB
График	розовая кривая «до», голубая «после»; пунктиры — $\pm\sigma$
Таблица	те же числа по каждому каналу

Режим **Values** для канала R:



Гистограмма значений канала R до и после фильтра

DENOISE FILTER

DENOISE FILTER

Gaussian

Median

Bilateral

weighted average with a bell-shaped kernel:
strongest smoothing, but edges get blurred too

Spatial σ 1.0 px

☐ Selected region only

Apply gaussian

R, G1, G2 and B are filtered separately, so colors
never mix. Ctrl+Z undoes the filter

Группа DENOISE FILTER

ЭЛЕМЕНТ	ЧТО ДЕЛАЕТ
Gaussian / Median / Bilateral	метод фильтра; под кнопками — краткое описание
Spatial σ	радиус сглаживания в пикселях канала (Gaussian и Bilateral)
Window per channel	окно медианы 3×3 или 5×5
Range σ и Auto	насколько разные значения считаются похожими (Bilateral); Auto ставит 2.5 σ шума выделенной области
Selected region only	фильтровать только выделение; без выделения фильтруется весь кадр
Apply	применить фильтр; во время работы — прогресс и Cancel

BEFORE REFERENCE

BEFORE REFERENCE

before gaussian $\sigma 1.0$

Capture current as before

Use original file as before

Lock reference

locked: every next filter is compared with the same reference, for example the original file

Группа BEFORE REFERENCE

С чем сравнивается «до»:

- **Capture current as before** — запомнить текущее состояние;
- **Use original file as before** — сравнивать с исходным файлом;
- **Lock reference** — не давать фильтрам перезаписывать «до».

Без блокировки перед каждым фильтром «до» запоминается автоматически.

Tab

 прячет панели, и график занимает всё окно:



Вкладка шума со скрытыми панелями

Теория, формулы и проверка фильтров — в разделе [Шум](#).

Строка состояния

- **Слева:** имя файла, размер в мегапикселях, узор CFA, black и white level, размер reference-кадра.
- **Справа:** название видеокарты или `no CUDA device`, занятая видеопамять, память программы, время последней операции, координаты и значение под курсором.

Горячие клавиши

КЛАВИША	ДЕЙСТВИЕ
<code>Ctrl + O</code>	открыть RAW
<code>Ctrl + S</code>	сохранить PNG
<code>Ctrl + W</code>	заккрыть файл
<code>Ctrl + Q</code>	выйти
<code>Ctrl + Z</code> / <code>Ctrl + Y</code>	отменить / вернуть
<code>Ctrl + D</code>	демозаик
<code>Ctrl + A</code>	добавить область в список
<code>Ctrl + 1</code> / <code>Ctrl + 2</code>	вкладка IMAGE / NOISE · FILTERS
<code>Ctrl + H</code>	вкладка шума
<code>Ctrl + Shift + D</code>	применить текущий фильтр шума
<code>Ctrl + Shift + C</code>	копировать вид
<code>G C T U M L B</code>	сетка, клиппинг, трети, линейки, миникарта, лупа, сравнение
<code>R</code> / <code>F</code> / <code>1</code>	поворот / вписать / 1:1
<code>F11</code>	полный экран
<code>Tab</code>	спрятать панели
стрелки / <code>Shift</code> +стрелки	сдвинуть область на 1 / 10 пикселей

Типичные сценарии

Шум ровного участка до и после фильтра

1. Открой файл и выдели ровный однотонный участок размером 100–500 пикселей.
2. `Ctrl + 2`, режим **Noise**.
3. Выбери фильтр и нажми **Apply**.
4. Посмотри на карточки и таблицу, затем `Ctrl + Z` и попробуй другой фильтр.
5. Чтобы все фильтры сравнивались с исходником: **Use original file as before**, затем **Lock reference**.

Временной шум и фиксированный узор

1. Сними два одинаковых кадра ровной сцены со штатива с одинаковыми настройками.
2. **Open RAW** — первый кадр, **Open reference RAW** — второй.
3. Выдели ровный участок и нажми **Compare with reference**.

Убрать горячие пиксели темновым кадром

1. Сними кадр с закрытой крышкой объектива с той же выдержкой, ISO и температурой.
2. Открой обычный кадр, reference — тёмный.
3. **Subtract reference as dark.** **Ctrl** + **Z** отменит вычитание.

Серия снимков по одним и тем же областям

1. Выдели области и добавь каждую через **Ctrl** + **A**.
2. **PRESET** → **Save**.
3. Открой следующий файл, **PRESET** → **Load**, затем **Export all regions to .csv**.

Цветная картинка

1. Выбери метод: **Malvar** — лучшее качество, **Binning** — самый быстрый.
2. **Ctrl** + **D**, подбери гамму.
3. **Ctrl** + **S** — PNG, или **Save linear to 16-bit TIFF** для точной обработки в других программах.

Несколько файлов

Каждый открытый файл получает вкладку над изображением. У каждого файла свои области, гамма, зум и поворот. Крестик на вкладке закрывает файл.

Шум

Этот раздел объясняет, что такое шум RAW-кадра, как программа его измеряет и как устроены фильтры шумоподавления. Все числа получены на реальных файлах `test.dng` (iPhone 8) и `test_2.DNG` (Pentax 645D) и на синтетическом шуме с известной σ .

Откуда берётся шум

Даже если снимать идеально ровную серую стену, соседние пиксели покажут разные значения. Основные причины:

ИСТОЧНИК	ОТКУДА	КАК СЕБЯ ВЕДЁТ
Фотонный (дробовой) шум	свет приходит отдельными фотонами, их число случайно	$\sigma = \sqrt{N}$ электронов: растёт с яркостью, но отношение сигнал/шум тоже растёт
Шум считывания	усилитель и АЦП сенсора	почти постоянен, заметнее всего в тенях
Темновой ток	электроны от нагрева сенсора	растёт с выдержкой и температурой, даёт горячие пиксели
Фиксированный узор (FPN)	разная чувствительность отдельных пикселей и столбцов	одинаков от кадра к кадру, поэтому его можно вычесть
Квантование	АЦП округляет до целого числа	± 0.5 ADU, важно только при очень малом шуме

Программа не разделяет все источники поштучно, но даёт инструменты для главных: σ области, двухкадровый метод (случайный шум против фиксированного узора), optical black (уровень чёрного и шум считывания) и темновой кадр.

Единицы и уровни

- **ADU** — «сырые» единицы сенсора, то число, которое записано в RAW.
- **Black level** — сколько ADU сенсор выдаёт в полной темноте. У `test.dng` — 528, у `test_2.DNG` — 0.
- **White level** — уровень насыщения: всё, что выше, пересвечено. У `test.dng` — 4095 (12 бит), у `test_2.DNG` — 15767.
- **SNR** — отношение сигнал/шум в децибелах:

$$\text{SNR} = 20 \cdot \log_{10}(\text{среднее} - \text{black level}) / \sigma$$

ТЕКСТ

Отношение 10 — это 20 dB, 31.6 — 30 dB, 100 — 40 dB.

Почему шум считается по каналам

Пиксели под красным, зелёным и синим фильтрами получают разное количество света, и их средние заметно различаются. Если смешать их в одну статистику, σ окажется завышенной: к шуму добавится разница между средними каналов. Поэтому программа всегда считает каналы **R**, **G1**, **G2**, **B** отдельно. Серая плашка из `test.dng`, область 256×256 с угла [3712, 1152]:

КАНАЛ	СРЕДНЕЕ, ADU	Σ , ADU
R	672.3	15.17
G1	738.1	19.96
G2	737.7	20.35
B	595.6	8.49

Насколько сильно смешивание завышает σ , видно на асфальте из `test_2.DNG` (область 256×256 с угла [2816, 3968]). У отдельных каналов σ от 43.65 до 75.09 ADU, а у всех пикселей вместе — 145.10 ADU, потому что средние каналов там сильно различаются: 393.9 у R и 713.9 у G1.

Для **ALL** на вкладке шума берётся объединённая σ без этой ошибки: $\sqrt{(\sigma_R^2 + \sigma_{G1}^2 + \sigma_{G2}^2 + \sigma_B^2) / 4}$.

Статистика области

Для каждого канала области программа считает:

```
среднее  $\mu = \Sigma v / N$   
 $\sigma = \sqrt{(\Sigma (v - \mu)^2 / N)}$   
min, max = наименьшее и наибольшее значение
```

ТЕКСТ

Ещё три числа — `range`, `useful` и `SNR` — описаны ниже, в разделе «Полезные биты».

σ делится на `N`, а не на `N - 1`. На области 256×256 в каждом канале 16 384 пикселя, и разница между этими вариантами меньше 0.01 %.

Статистика считается в два параллельных прохода (функция `compute_region_channels`): первый собирает суммы, минимумы и максимумы, второй — квадраты отклонений от уже известного среднего. Каждый поток копит результат в своих локальных переменных и отдаёт его только в конце, поэтому потоки не мешают друг другу в кэше процессора.

Полезные биты

Кроме шума в ADU программа показывает три числа, которые переводят шум на язык разрядности. Все они считаются отдельно для каждого канала, для выделенной области.

```
range = log2(max - min + 1)  
useful = log2((white - black) /  $\sigma$ )
```

ТЕКСТ

$$\text{SNR} = 20 \cdot \log_{10}((\text{среднее} - \text{black}) / \sigma)$$

число	что означает
range bits	сколько бит занимает разброс значений в области. На ровном участке это разброс самого шума, на пёстром — разброс сюжета
useful bits	сколько уровней различимо над шумом: полный диапазон сенсора делится на σ . Это и есть «полезные биты»
SNR	отношение сигнал/шум в децибелах; 6.02 dB — это ровно один бит

Раньше программа показывала только $\log_2(\text{max} - \text{min})$, то есть размах значений. Это число не говорит ничего о качестве: на шумном участке оно большое именно из-за шума. Полезные биты устроены иначе — чем меньше шум, тем их больше.

Пример. Сенсор Pentax 645D: black 0, white 15767, то есть полный диапазон 15767 ADU. На асфальте σ канала B равна 67.49 ADU:

$$\text{useful} = \log_2(15767 / 67.49) = \log_2(233.6) = 7.87 \text{ бита}$$

ТЕКСТ

Из 14 бит АЦП на этом участке реально различимы примерно 8. Остальное съедено шумом и фактурой асфальта.

Связь с ENOB. В технике АЦП принято считать эквивалентное число разрядов с поправкой на шум квантования: $\text{ENOB} = \log_2(\text{FS} / (\sigma \cdot \sqrt{12})) = \text{useful} - 1.79$. Программа показывает `useful`, вычесть 1.79 при необходимости просто.

Что портит результат. σ должна быть шумом, а не сюжетом. На текстуре полезные биты получаются заниженными. Поэтому меряй на ровном однотонном участке, как описано ниже.

Динамический диапазон всего сенсора считается по шуму считывания: выполни **Measure black from optical area**, и программа покажет $\log_2((\text{white} - \text{black}) / \sigma)$ по закрытой зоне. Это верхняя граница: лучше этого числа от камеры не получить.

Как правильно мерить шум

1. **Ровный однотонный объект:** серая карта, лист бумаги, ровное небо. На текстуре (асфальт, ткань) в σ попадает сама текстура.
2. **Без пересвета:** максимум области должен быть ниже white level. Проверь режимом **Clipping** (C).
3. **Размер 100×100 ... 500×500:** меньше — числа скачут, больше — попадает неравномерность освещения и виньетирование.
4. **Проверка:** профиль области почти ровный, а гистограмма **Values** для каждого канала — один узкий колокол.

Серая плашка `test.dng` даёт σ 8–20 ADU, асфальт в `test_2.DNG` — 44–75 ADU: к шуму там добавилась фактура асфальта.

Гистограмма и профиль

Histogram (панель ANALYSIS) делит диапазон 0...white level на 256 столбцов и считает, сколько пикселей каждого канала попало в каждый столбец. **Profile** показывает среднее по каждому столбцу области, а для области шириной в один пиксель — по каждой строке.

Гистограммы до и после



Вкладка NOISE · FILTERS: асфальт test_2.DNG после Gaussian σ 1.0, σ 66.20 → 44.27

Режим Noise

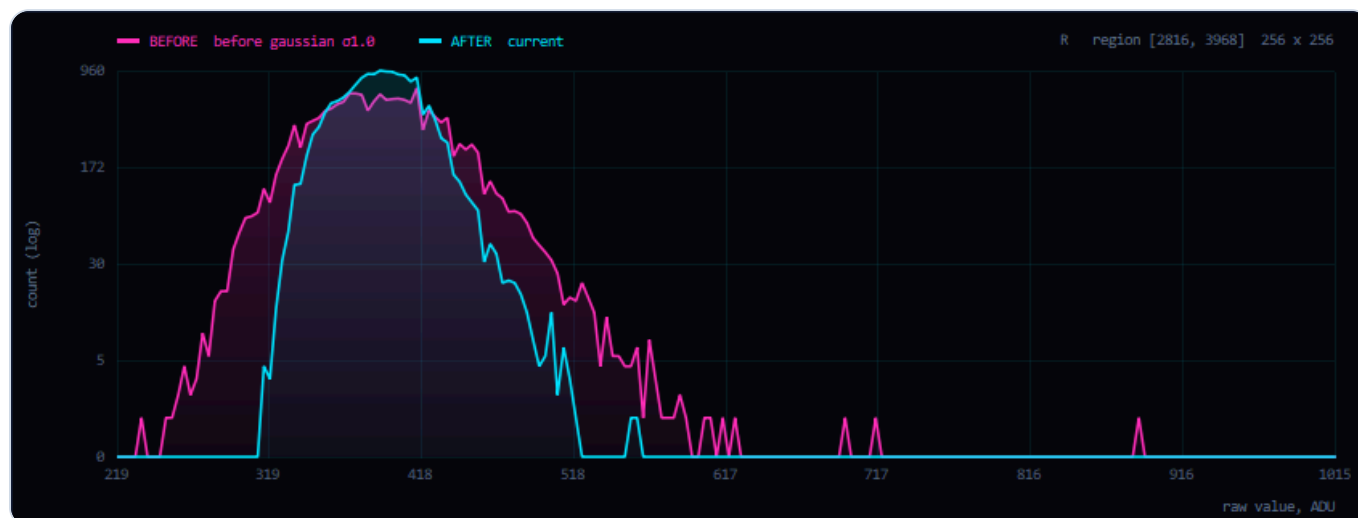
Для каждого пикселя считается отклонение от среднего его канала: $v - \mu$. Это «форма шума» без учёта уровня яркости.

- **Диапазон** — $\pm 4\sigma$ самого шумного канала снимка «до».
- **Ширина столбца** — целое число ADU (1, 2, 3...). Значения RAW целые, и при дробной ширине одни столбцы получали бы одно значение, а другие два. Гистограмма превращалась бы в «гребёнку».
- **«До» и «после» строятся на одинаковых столбцах**, поэтому кривые можно сравнивать напрямую.

- **Пунктиры $\pm\sigma$:** розовые — «до», голубые — «после». Чем ближе голубые к центру, тем меньше шум.

Режим Values

Гистограмма самих значений в диапазоне среднее $\pm 4\sigma$. Видно, где лежит каждый канал и насколько сузился его колокол.



Режим Values, канал R

С чем сравнивается «до»

- По умолчанию «до» — исходный файл.
- Перед каждым фильтром, темновым кадром или сменой black level текущее состояние запоминается как «до» автоматически.
- **Capture current as before** запоминает вручную.
- **Use original file as before** возвращает сравнение с исходником.
- **Lock reference** запрещает перезапись. Так цепочка из нескольких фильтров сравнивается с одним и тем же исходником.

«До» хранится как полная копия кадра. Поэтому при смене выделенной области гистограмма «до» пересчитывается для той же области.

Двухкадровый метод

Нужны два снимка одной и той же ровной сцены с одинаковыми настройками, лучше со штатива. Фиксированный узор одинаков на обоих кадрах, а случайный шум разный. Если вычесть кадры, узор исчезнет, а случайный шум останется.

$D = A - B$	разность кадров, пиксель за пикселем
$rd = \sqrt{\text{Var}(D) / 2}$	временной (случайный) шум одного кадра
$tot = \sqrt{\text{Var}(A)}$	полный шум кадра A
$fpn = \sqrt{\max(0, \text{Var}(A) - \text{Var}(D)/2)}$	фиксированный узор

ТЕКСТ

Деление на 2 нужно потому, что при вычитании двух независимых кадров их случайные дисперсии складываются: $\text{Var}(D) = 2 \cdot \text{rd}^2$. Полный шум складывается из случайного и постоянного: $\text{tot}^2 = \text{rd}^2 + \text{fpn}^2$. Всё считается по каналам R, G1, G2, B внутри выделенной области или по всему кадру.

Как читать результат. Если `fpn` мал по сравнению с `rd`, шум почти целиком случайный, и его снижают усреднением кадров и фильтрами. Если `fpn` велик, помогут темновой кадр или калибровка плоским полем.

Темновой кадр

Кадр, снятый с закрытой крышкой объектива с той же выдержкой, ISO и температурой, содержит темновой ток, горячие пиксели и узор. Программа вычитает его из обычного кадра:

```
результат = кадр - тёмный кадр + black level      (обрезается в 0...white level)
```

ТЕКСТ

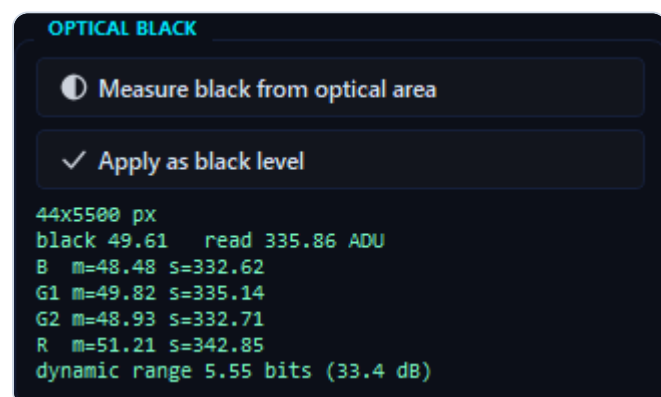
Black level добавляется обратно, потому что тёмный кадр сам содержит black level. Иначе результат ушёл бы ниже нуля.

ЗАМЕТКА

Случайный шум тёмного кадра при вычитании добавляется к шуму результата. Если шумы кадров одинаковы, случайная часть вырастет в $\sqrt{2}$ раза. Поэтому в астро- и научной съёмке вычитают среднее из нескольких тёмных кадров. Программа вычитает один кадр.

Optical black

У многих камер по краю сенсора есть полоса пикселей, закрытых от света. Программа ищет её слева от активной области (если отступ не меньше 6 пикселей), иначе сверху. Среднее этой зоны — реальный уровень чёрного, а σ — шум считывания.



Результат измерения на Pentax 645D

У `test_2.DNG` найдена полоса 44×5500 пикселей: среднее 49.61 ADU, но $\sigma = 335.86$ ADU.

Настоящая закрытая зона даёт σ в единицы ADU, так что эта полоса светится, и применять её как black level нельзя. У `test.dng` (iPhone 8) такой зоны нет.

Apply as black level записывает округлённое среднее в black level кадра. Это отменяется через `Ctrl + Z`.

Фильтры шумоподавления

Общая схема

Все три фильтра работают по одной последовательности (функция `denoise_frame`):

1. **Область с запасом.** К выделенной области добавляется поле `2 · (r + 1)` пикселей (r — радиус фильтра), чтобы на границе выделения не было ступеньки. Начало выравнивается на чётные координаты, чтобы не сбить узор Байера.
2. **Разделение на каналы.** Область раскладывается на 4 плоскости половинного размера — R, G1, G2 и B. Цвета не смешиваются.
3. **Фильтр каждой плоскости.** Строки делятся между потоками. За краем плоскости подставляется ближайший крайний пиксель (структура `PlaneIndex`).
4. **Возврат в кадр.** Отфильтрованные значения записываются только внутрь выделения и округляются до целых ADU.
5. **Прогресс и отмена.** Главный поток раз в 2 мс обновляет процент. Если нажать **Cancel**, кадр остаётся без изменений.

ВАЖНО

Spatial σ задаётся в пикселях **плоскости канала**. Соседние пиксели одного цвета на сенсоре стоят через один, поэтому 1 пиксель плоскости = 2 пикселя сенсора.

Gaussian

Взвешенное среднее соседей с весами в форме «колокола»:

$$w(i) = \exp(-i^2 / (2\sigma^2)) / \sum \exp(\dots) \quad i = -r \dots r, \quad r = \lceil 3\sigma \rceil$$

ТЕКСТ

Фильтр **разделимый**: сначала проход по строкам, затем по столбцам. Для $\sigma = 1$ это 7 + 7 умножений на пиксель вместо 49 для окна 7×7, а результат тот же.

Чего ждать. Для некоррелированного шума двумерный гауссов фильтр уменьшает σ в `2σ/π` раз. При $\sigma = 1$ это в 3.54 раза: шум 20 ADU должен стать 5.64 ADU. Края тоже размываются.

Median

Значения окна 3×3 (9 пикселей) или 5×5 (25 пикселей) частично упорядочиваются, и берётся средний по порядку элемент. Это делает `std::nth_element` за линейное время, без полной сортировки.

Чего ждать. Для гауссова шума медиана из n значений даёт σ примерно в $\sqrt{\pi / 2n}$ от исходной: 0.418 для 3×3 и 0.251 для 5×5. Медиана устойчива к выбросам, поэтому лучше всех убирает горячие пиксели и одиночные точки, а резкие края сохраняет.

Bilateral

Как гауссов фильтр, но вес соседа зависит ещё и от того, насколько его значение похоже на центральное:

$$w = \exp\left(-\frac{dx^2 + dy^2}{2\sigma_s^2}\right) \cdot \exp\left(-\frac{(v_{\text{сосед}} - v_{\text{центр}})^2}{2\sigma_r^2}\right)$$
$$r = [2\sigma_s]$$

ТЕКСТ

- σ_s (Spatial σ) — насколько далеко смотреть.
- σ_r (Range σ) — какая разница значений ещё считается «тем же объектом».
- **Сосед на другой стороне края** отличается сильно, получает почти нулевой вес и не размывает край.

Веса по расстоянию и таблица весов по разнице значений (до $6\sigma_r$) считаются заранее, поэтому в цикле нет вызовов `exp`.

Как выбрать σ_r . Примерно 2–3 σ шума ровного участка; кнопка **Auto** ставит 2.5 σ . Если σ_r слишком велик, фильтр работает как обычный Gaussian и размывает края. Если слишком мал, он почти ничего не делает.

Проверка на синтетическом шуме

Кадр 1001×801 с уровнями каналов 1000 / 2000 / 3000 / 4000 ADU и гауссовым шумом $\sigma = 20$ ADU. Фильтр применён ко всему кадру.

ФИЛЬТР	Σ ПОСЛЕ	ТЕОРИЯ	СРЕДНИЕ КАНАЛОВ
Gaussian σ 1.0	5.68	5.64	без изменений
Median 3×3	8.17	8.36	без изменений
Median 5×5	4.99	5.02	без изменений
Bilateral σ_s 1.0, σ_r 40	8.51	—	без изменений

Дополнительно проверено:

- в режиме «только область» пиксели за её пределами не меняются;
- после отмены кадр остаётся прежним;
- область у самого края кадра обрабатывается без ошибок.

Через интерфейс программы тот же тест даёт σ 19.99 → 5.63 для Gaussian σ 1.0:



Синтетический тест в программе: шум σ 20 ADU после Gaussian σ 1.0

На реальных кадрах

Gaussian σ 1.0, область 256×256 (docs/examples/noise_report.cpp):

КАНАЛ	TEST.DNG , СЕРАЯ ПЛАШКА	TEST_2.DNG , АСФАЛЬТ
R	15.17 → 12.26 (-19.2 %)	43.65 → 27.58 (-36.8 %)
G1	19.96 → 17.18 (-13.9 %)	75.09 → 49.66 (-33.9 %)
G2	20.35 → 17.79 (-12.5 %)	73.70 → 48.37 (-34.4 %)
B	8.49 → 6.00 (-29.4 %)	67.49 → 47.67 (-29.4 %)

На реальных снимках эффект заметно слабее, чем на синтетике, и это ожидаемо:

- **iPhone** сохраняет в DNG уже частично обработанный сигнал; шум соседних пикселей связан между собой, и усреднение помогает хуже;
- **на асфальте** в σ входит фактура поверхности, которую фильтр не должен и не может убрать.

Скорость

Весь кадр 4032×3024, Release-сборка, AMD Ryzen 5 7500F (12 потоков):

ФИЛЬТР	ВРЕМЯ
Gaussian σ 1.0	156 мс
Median 3×3	185 мс
Median 5×5	413 мс
Bilateral σ s 1.0	167 мс

В Debug-сборке те же фильтры работают в 3.4–6.2 раза медленнее.

Какой фильтр выбрать

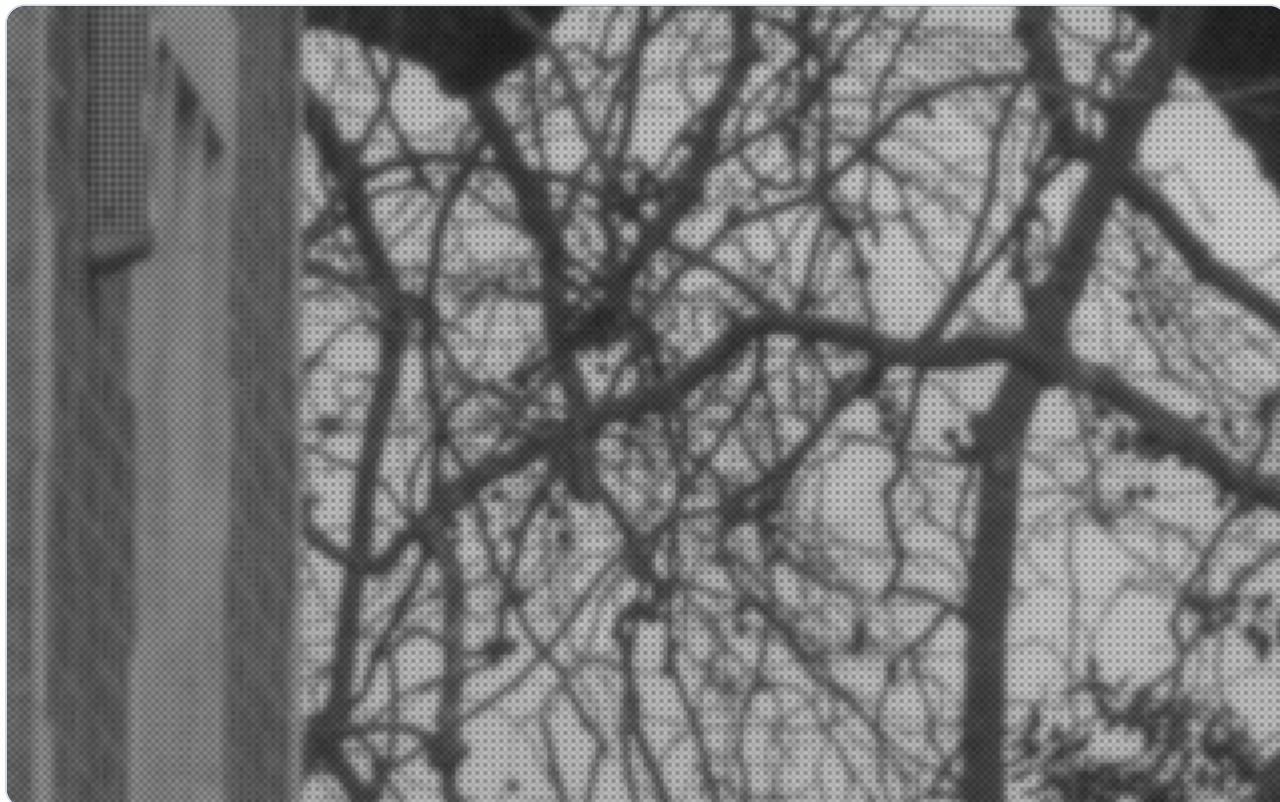
ЗАДАЧА	ФИЛЬТР
Максимально снизить σ на ровном участке	Gaussian σ 1–2
Убрать горячие пиксели и одиночные выбросы	Median 3×3
Сильнее убрать шум и сохранить края объектов	Bilateral, Range σ через Auto
Сохранить мелкие детали	Median 3×3 или Bilateral с небольшим Range σ

Цвет

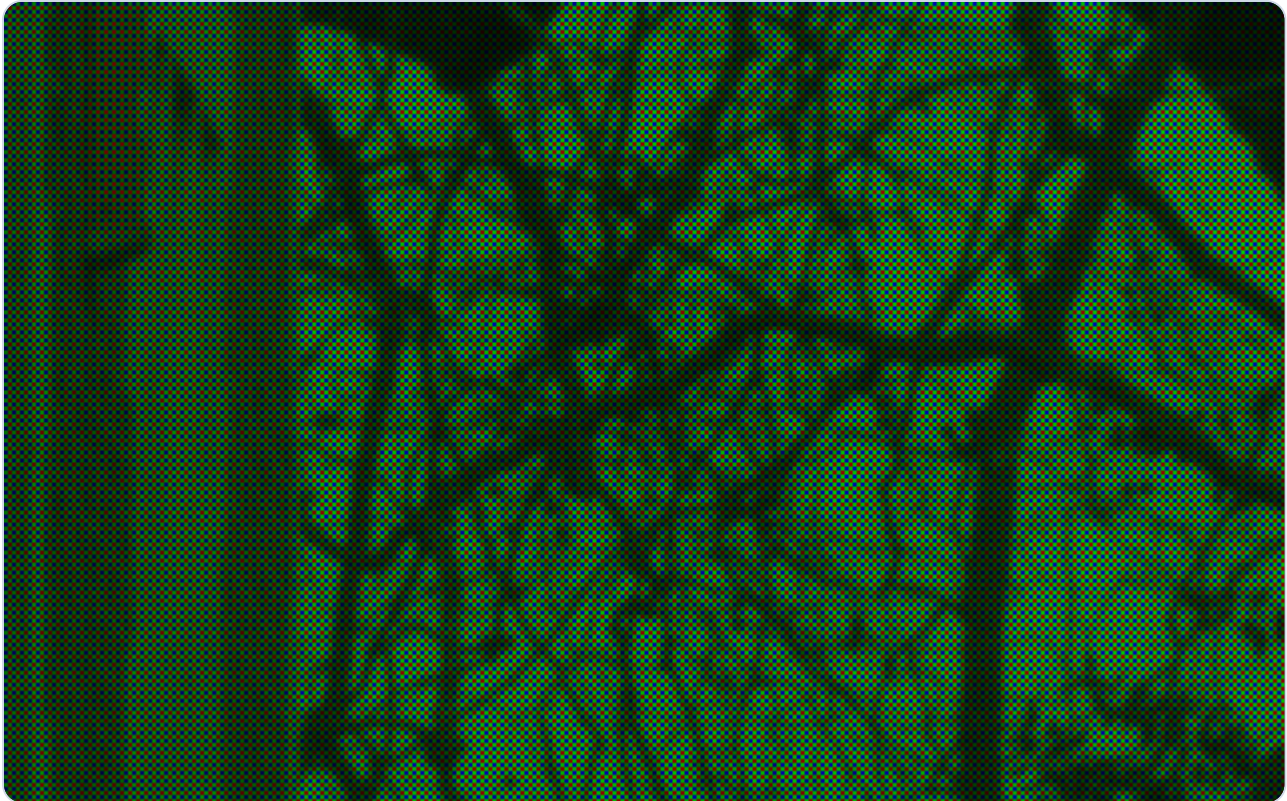
Сенсор камеры не видит цвет. Над каждым пикселем стоит цветной фильтр, и пиксель измеряет яркость только одного цвета: красного, зелёного или синего. Восстановить полноценный цвет в каждой точке — задача **демозаика** (demosaicing). В программе три метода: Binning 2×2, Bilinear и Malvar-He-Cutler.

Мозаика Байера

Фильтры повторяются блоком 2×2: один красный, один синий и два зелёных. Зелёных вдвое больше, потому что глаз сильнее всего чувствителен к зелёному, и именно зелёный канал несёт основную резкость.



RAW-данные до демозаика: один канал на пиксель, для наглядности с гаммой 2.2. Кноп 320×200 из кадра Pentax 645D, увеличение ×2



Тот же фрагмент, где каждый пиксель окрашен в цвет своего фильтра. Зелёных пикселей вдвое больше, поэтому картинка зелёная

У разных камер блок начинается с разного цвета. Программа определяет узор автоматически и показывает его в строке состояния и в HUD:

УЗОР	ВЕРХНИЙ РЯД	НИЖНИЙ РЯД	ПРИМЕР
RGGB	R G	G B	iPhone 8, test.dng
BGGR	B G	G R	Pentax 645D, test_2.DNG
GRBG	G R	B G	—
GBRG	G B	R G	—

Как узор хранится в коде

LibRaw отдаёт узор 32-битным числом `filters` (в программе `RawImage::cfaPattern`). На каждую позицию отведено 2 бита: 0 — красный, 1 — зелёный, 2 — синий, 3 — второй зелёный. Цвет пикселя `(x, y)` вычисляет `RawImage::color_at`:

```
int color_at(int x, int y) const {  
    return (cfaPattern >> (((y << 1) & 14) | (x & 1)) << 1)) & 3;  
}
```

C++

Для `test.dng` это `0xb4b4b4b4` (RGGB), для `test_2.DNG` — `0x1e1e1e1e` (BGGR). Функция `cfa_index(x, y) = ((y & 1) << 1) | (x & 1)` нумерует позиции внутри блока 2×2 числами 0...3. По этим номерам вся статистика и фильтры разделяют пиксели на каналы R, G1, G2, B.

Подготовка значений

Все три метода начинают одинаково.

1. **Нормировка.** Значение сенсора переводится в диапазон 0...1:

```
v = (raw - black_level) / (white_level - black_level)
```

ТЕКСТ

`black_level` — уровень, который сенсор выдаёт в полной темноте, `white_level` — уровень насыщения. У `test.dng` это 528 и 4095, у `test_2.DNG` — 0 и 15767.

2. **Баланс белого.** Красный и синий умножаются на коэффициенты камеры из файла (`cam_mul`), поделённые на зелёный. У `test.dng` : R × 1.552, B × 2.841. У `test_2.DNG` : R × 1.855, B × 1.258. После этого серое в сцене становится серым на картинке.

3. **Ограничение и 8 бит.** Результат обрезается в 0...1 и умножается на 255.

ЗАМЕТКА

Программа не применяет цветовую матрицу камеры (перевод из «RGB камеры» в sRGB) и не восстанавливает пересвеченные участки. Цвета получаются правдоподобными, но не колориметрически точными. Для анализа шума это не важно: статистика и фильтры работают с исходными RAW-значениями.

Binning 2×2

Самый простой метод. Каждый блок 2×2 превращается в **один** пиксель:

```
R = значение красного пикселя блока  
B = значение синего пикселя блока  
G = (G1 + G2) / 2
```

ТЕКСТ

Картинка получается **вдвое меньше по каждой стороне**, зато в каждой точке все три цвета измерены, а не придуманы. Усреднение двух зелёных ещё и немного снижает шум зелёного канала.



Binning 2×2: тот же фрагмент, увеличение ×4. Видны крупные квадраты и цветные ореолы на тонких ветках

Bilinear

Полный размер: недостающие цвета в каждом пикселе берутся **средним ближайших соседей** того же цвета.

ГДЕ СТОИТ ПИКСЕЛЬ	ЗЕЛЁНЫЙ	КРАСНЫЙ / СИНИЙ
красный или синий фильтр	среднее 4 соседей по горизонтали и вертикали	недостающий цвет — среднее 4 соседей по диагонали
зелёный фильтр	своё значение	цвет соседей в строке — среднее левого и правого, цвет соседей в столбце — среднее верхнего и нижнего

Какой из двух цветов лежит в строке, программа узнаёт по цвету левого соседа (`color_at(x - 1, y)`).



Bilinear: полный размер, увеличение $\times 2$. Плавно, но мелкие детали размыты

Malvar-He-Cutler

Метод из статьи Malvar, He, Cutler (Microsoft Research, 2004). Он тоже линейный, но добавляет **поправку по градиенту**: смотрит, как меняется яркость в самом пикселе и на расстоянии двух пикселей, и корректирует интерполяцию. Края получаются резче, цветных ореолов меньше, а считается почти так же быстро, как Bilinear.

Каждое значение — взвешенная сумма окна 5×5 , делённая на 8. Обозначения: v — пиксель в центре, $\Sigma_{\text{орт}}$ — сумма 4 соседей по горизонтали и вертикали, $\Sigma_{\text{диаг}}$ — 4 соседей по диагонали, $\Sigma_{\text{даль}}$ — 4 пикселей на расстоянии 2.

Зелёный в красном или синем пикселе:

$$G = (4 \cdot v + 2 \cdot \Sigma_{\text{орт}} - \Sigma_{\text{даль}}) / 8$$

ТЕКСТ

$$\begin{array}{ccccc} 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ -1 & 2 & 4 & 2 & -1 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{array} \times 1/8$$

Синий в красном пикселе (и красный в синем):

$$B = (6 \cdot v + 2 \cdot \Sigma_{\text{диаг}} - 1.5 \cdot \Sigma_{\text{даль}}) / 8$$

ТЕКСТ

$$\begin{array}{ccccc} 0 & 0 & -1.5 & 0 & 0 \\ 0 & 2 & 0 & 2 & 0 \end{array}$$

```
-1.5  0  6  0 -1.5  × 1/8
  0  2  0  2  0
  0  0 -1.5  0  0
```

Цвет соседей по строке в зелёном пикселе:

```
C = (5·v + 4·Σгор - Σдиаг - Σдаль_гор + 0.5·Σдаль_верт) / 8
```

ТЕКСТ

```
  0  0  0.5  0  0
  0 -1  0  -1  0
-1  4  5  4 -1  × 1/8
  0 -1  0  -1  0
  0  0  0.5  0  0
```

Для цвета соседей по столбцу берётся та же маска, повернутая на 90°.



Malvar-He-Cutler: полный размер, увеличение ×2. Ветки и кирпичи чётче, чем у Bilinear

Края кадра. Когда окну не хватает пикселей у границы, `raw_at` подставляет ближайший пиксель внутри кадра. Поэтому по краям нет чёрной рамки.

CPU и GPU

	CPU	GPU (CUDA)
Где считается	<code>demosaic_cpu_progress</code> , <code>demosaic_full_progress</code>	ядра <code>demosaicBinKernel</code> , <code>demosaicFullKernel</code>

	CPU	GPU (CUDA)
Параллельность	строки делятся на полосы, каждая полоса в своём потоке (<code>std::async</code>); потоков — по числу ядер, но не больше чем по одному на 64 строки	каждый пиксель — отдельный поток GPU, блоки 16×16
Прогресс и отмена	главный поток раз в 2 мс спрашивает <code>report</code> ; если тот вернул <code>false</code> , работа останавливается	кадр считается полосами: 256 строк для Binning, 512 для полного размера; после каждой полосы — <code>report</code>
Данные	в памяти процесса	загружаются в видеопамять один раз, результат скачивается в конце

Гамма

Линейные значения сенсора выглядят тёмными, потому что глаз воспринимает яркость нелинейно. После демозаика программа применяет гамму:

```
out = 255 * (in / 255)^(1/γ), по умолчанию γ = 2.2
```

ТЕКСТ

Гамма меняет только вид цветной картинки и PNG; RAW-данные, статистику и фильтры она не трогает. На CPU гамма считается в одном потоке (`apply_gamma`), на GPU — ядром `applyGammaKernel` по 256 потоков в блоке.

Сравнение методов

Замеры `docs/examples/demosaic_benchmark.cpp` , Release-сборка. Время включает демозаик и гамму. Процессор AMD Ryzen 5 7500F (12 потоков), видеокарта NVIDIA GeForce RTX 3080 Ti.

МЕТОД	РАЗМЕР	TEST.DNG 4032×3024, CPU	GPU	TEST_2.DNG 7328×5501, CPU	GPU
Binning 2×2	½ × ½	77.9 мс	7.5 мс	254.2 мс	21.2 мс
Bilinear	полный	312.5 мс	17.3 мс	1012.8 мс	56.4 мс
Malvar-He-Cutler	полный	317.1 мс	17.6 мс	1038.5 мс	60.3 мс

Сам демозаик на CPU без гаммы идёт намного быстрее: 6 / 20 / 34 мс для `test.dng` и 17 / 64 / 88 мс для `test_2.DNG` . Большую часть CPU-времени занимает однопоточная гамма.

	BINNING 2×2	BILINEAR	MALVAR-HE-CUTLER
Размер результата	вдвое меньше	полный	полный
Резкость	низкая (меньше пикселей)	средняя, детали размыты	лучшая из трёх
Цветные артефакты	ореолы на тонких линиях	«молния» (zipper) на резких краях	замечто меньше

	BINNING 2×2	BILINEAR	MALVAR-HE-CUTLER
Когда выбирать	быстрый предпросмотр, большие файлы	простая наглядная интерполяция	итоговая картинка

Режим **Compare before / after** (B) показывает RAW и результат рядом, разделитель можно тянуть мышью:



Сравнение: слева RAW, справа результат Malvar

Форматы файлов

КОМАНДА	ФАЙЛ	ЧТО ВНУТРИ
Save RAW to .Bin	raw_7328x5501_uint16.bin	значения сенсора как есть: uint16, little-endian, без заголовка, строка за строкой; размер = ширина × высота × 2 байта
Save region to .Bin	region_256x256_uint16.bin	то же самое, но только выделенная область
Save RAW to .TIFF	raw_mosaic.tiff	та же мозаика, 16-битный серый TIFF
Save region to .TIFF	region_256x256_uint16.tiff	мозаика выделенной области, 16-битный серый TIFF
Save to .png	output.png	цветная картинка после демозаика и гаммы, 8 бит на канал
Save linear to 16-bit TIFF	linear16.tiff	цвет без гаммы, 16 бит на канал, метод Binning (половинный размер)

Каждое сохранение кладётся в **отдельную папку**, названную по имени файла, и папка создаётся сама. Выбрали C:/data/region_256x256_uint16.bin — получите:

```
C:/data/region_256x256_uint16/  
region_256x256_uint16.bin  
region_256x256_uint16_mathcad.txt
```

ТЕКСТ

Рядом с `.bin` лежит `имя_mathcad.txt`, рядом с `.tiff` — `имя_info.txt`. Это обычный текст, его открывает блокнот:

```
M := READBIN("C:/data/region_256x256_uint16/region_256x256_uint16.bin", "uint16", 0, 256, 0, 256)
width      256
height     256
type       uint16 little-endian
bytes      131072
black level 0
white level 15767
cfa        BGR
camera     Pentax 645D
```

ТЕКСТ

Первая строка — готовая команда для Mathcad, её достаточно скопировать. У `.tiff` этой строки нет: размеры уже записаны в заголовке самого TIFF.

ВАЖНО

В самом `.bin` нет заголовка, там только числа. Программа, которая его читает, обязана знать ширину и высоту. Если указать не ту ширину, строки сдвинутся и картинка поедет по диагонали, будто смазана. Если перепутать ширину с высотой, изображение рассыплется совсем. Первая проверка простая: размер файла должен быть ровно `ширина × высота × 2` байта.

Как прочитать в Python

```
import numpy as np

raw = np.fromfile("raw_7328x5501_uint16.bin", dtype="<u2").reshape(5501, 7328)

print(raw.shape, raw.min(), raw.max(), raw.mean())
```

PYTHON

Ширину и высоту возьмите из `имя_mathcad.txt` — или прямо из имени файла, программа всегда вписывает туда размер.

Готовый просмотрщик с ползунком гаммы лежит в `test.py`, он сам читает размеры из `имя_mathcad.txt`:

```
python test.py raw_7328x5501_uint16.bin
```

POWERSHELL

Как прочитать в Mathcad

Программа сама пишет готовую строку первой строкой файла `имя_mathcad.txt` рядом с `.bin` — её достаточно скопировать в Mathcad. Устроена она так:

```
M := READBIN("C:/data/region_256x256_uint16.bin", "uint16", 0, 256, 0, 256)
```

ТЕКСТ

АРГУМЕНТ	В ПРИМЕРЕ	ЧТО ОЗНАЧАЕТ
файл	"C:/data/region_256x256_uint16.bin"	полный путь; косые черты пишутся как /
тип	"uint16"	16-битные целые без знака — именно так пишет программа
endian	0	порядок байтов little-endian (1 — big-endian)
cols	256	число столбцов в строке файла, то есть ширина области
skip	0	сколько байт пропустить в начале файла
maxrows	256	сколько строк прочитать, то есть высота

Дальше `M` — обычная матрица: работают `rows(M)`, `cols(M)`, `mean(M)`, `stdev(M)`, `max(M)`, `min(M)`.

Значения лежат в мозаике Байера, поэтому соседние числа относятся к разным каналам. Чтобы получить один канал, бери каждый второй пиксель по строке и по столбцу. Для узора `BGGR` в левом верхнем углу стоит B, значит чётные строки и чётные столбцы — синий канал:

```
f(i, j) := M[2i, 2j
B := matrix(rows(M) / 2, cols(M) / 2, f)
```

ТЕКСТ

Первая буква строки `cfa` в текстовом блокноте — это как раз канал пикселя (0, 0).

ВНИМАНИЕ
Целый кадр в Mathcad открывать не стоит: 7328 × 5501 — это 40 миллионов чисел, и программа зависнет. Выдели область мышью и сохрани её кнопкой **Save region to .Bin** в группе EXPORT RAW. Область 256 × 256 — это 65 536 чисел, с такой матрицей Mathcad считает нормально.

Справочник функций

Вся обработка написана как библиотека из заголовочных файлов: функции объявлены `inline` прямо в `.hpp`. Чтобы использовать их в своей программе, подключи нужный заголовок и собери программу вместе с LibRaw и libtiff, а для функций GPU — ещё и с CUDA.

ЗАГОЛОВОК	ЧТО ВНУТРИ
<code>im_read.hpp</code>	типы <code>Image</code> , <code>Image16</code> , <code>RawImage</code> ; загрузка RAW; демозаик на CPU; сохранение файлов; гамма
<code>image_stats.hpp</code>	статистика, гистограммы, профиль, двухкадровый шум, темновой кадр
<code>denoise.hpp</code>	фильтры шумоподавления по CFA-каналам
<code>process.hpp</code>	выбор устройства и метода демозаика (подключает <code>gpu_process.hpp</code>)
<code>gpu_process.hpp</code>	CUDA: память, запуск ядер, информация об устройстве
<code>im_process.cuh</code>	объявления функций, запускающих CUDA-ядра из <code>im_process.cu</code>

ВАЖНО

`im_read.hpp` содержит реализацию библиотеки `stb_image`. Подключай его (напрямую или через другие заголовки) только в **один** `.cpp` каждой программы, иначе линковщик выдаст ошибку LNK2005.

Как собрать свою программу

Только CPU

В консоли **x64 Native Tools Command Prompt for VS 2022**:

```
cl /O2 /MD /EHsc /std:c++17 /utf-8 /DNOMINMAX ^
/I"C:\Users\RED.DOT\Desktop\PNG NOISE" ^
/IC:\vcpkg\installed\x64-windows\include /IC:\vcpkg\installed\x64-windows\include\libraw ^
my_tool.cpp ^
/link C:\vcpkg\installed\x64-windows\lib\raw_r.lib C:\vcpkg\installed\x64-windows\lib\tiff.lib ^
C:\vcpkg\installed\x64-windows\lib\lcms2.lib C:\vcpkg\installed\x64-windows\lib\z.lib ws2_32.lib
```

CMD

Перед запуском добавь в `PATH` папку с DLL: `set PATH=C:\vcpkg\installed\x64-windows\bin;%PATH%`.

С GPU

Проще всего добавить цель в `CMakeLists.txt` рядом с существующими:

```
add_executable(my_tool my_tool.cpp im_process.cu)
target_include_directories(my_tool PRIVATE ${CMAKE_CURRENT_SOURCE_DIR})
target_link_libraries(my_tool PRIVATE CUDA::cudart libraw::raw_r TIFF::TIFF)
```

Готовые примеры

Оба примера лежат в `docs/examples/`, собраны и проверены. Вывод ниже — настоящий.

noise_report.cpp

Считает шум области по каналам, применяет Gaussian σ 1.0, сравнивает σ до и после и сохраняет цветную картинку Malvar.

C++

```
#include "im_read.hpp"
#include "image_stats.hpp"
#include "denoise.hpp"

#include <cstdio>
#include <cstdlib>

int main(int argc, char** argv) {

    if (argc < 2) {
        std::printf("usage: noise_report <file.dng> [x y w h]\n");
        return 1;
    }

    std::optional<RawImage> raw = load_raw(argv[1]);
    if (!raw) return 1;

    int x = argc > 5 ? std::atoi(argv[2]) : raw->width / 2 - 128;
    int y = argc > 5 ? std::atoi(argv[3]) : raw->height / 2 - 128;
    int w = argc > 5 ? std::atoi(argv[4]) : 256;
    int h = argc > 5 ? std::atoi(argv[5]) : 256;

    std::printf("%dx%d CFA %s black %d white %d region [%d, %d] %dx%d\n",
        raw->width, raw->height, cfa_pattern_name(*raw).c_str(),
        raw->black_level, raw->white_level, x, y, w, h);

    ChannelStats before = compute_region_channels(*raw, x, y, w, h);

    DenoiseParams params;
    params.method = DenoiseMethod::Gaussian;
    params.sigma = 1.0;

    RawImage filtered = *raw;

    denoise_frame(filtered, x, y, w, h, params, {});

    ChannelStats after = compute_region_channels(filtered, x, y, w, h);

    for (int k = 0; k < 4; ++k) {

        double change = (after.stats[k].stdDev - before.stats[k].stdDev) / before.stats[k].stdDev *
            100.0;

        std::printf("%-2s mean %7.1f sigma %6.2f → %6.2f (%+.1f%%)\n",
```

```

        before.labels[k].c_str(), before.stats[k].mean,
        before.stats[k].stdDev, after.stats[k].stdDev, change);
    }

    Image color = demosaic_full_progress(filtered, true, {});

    save_image("malvar_gamma.png", apply_gamma(color, 2.2f));
    save_raw_in_tiff("filtered_mosaic.tiff", filtered);

    std::printf("saved malvar_gamma.png and filtered_mosaic.tiff\n");
    return 0;
}

```

```

> noise_report.exe test.dng 3712 1152 256 256
4032x3024 CFA RGGB black 528 white 4095 region [3712, 1152] 256x256
R mean 672.3 sigma 15.17 → 12.26 (-19.2%)
G1 mean 738.1 sigma 19.96 → 17.18 (-13.9%)
G2 mean 737.7 sigma 20.35 → 17.79 (-12.5%)
B mean 595.6 sigma 8.49 → 6.00 (-29.4%)
saved malvar_gamma.png and filtered_mosaic.tiff

```

ТЕКСТ

demosaic_benchmark.cpp

Замеряет три метода демозаика на CPU и GPU и сохраняет результат Malvar.

```

#include "process.hpp"

#include <chrono>
#include <cstdio>

int main(int argc, char** argv) {

    if (argc < 2) {
        std::printf("usage: demosaic_benchmark <file.dng>\n");
        return 1;
    }

    std::optional<RawImage> raw = load_raw(argv[1]);
    if (!raw) return 1;

    bool gpu = cuda_device_available();

    std::printf("%dx%d device: %s\n", raw->width, raw->height, cuda_device_name().c_str());

    if (gpu) process_demosaic(*raw, Backend::GPU, DemosaicMethod::Binning, {});

    const DemosaicMethod methods[] = { DemosaicMethod::Binning, DemosaicMethod::Bilinear,
    DemosaicMethod::Malvar };
    const Backend backends[] = { Backend::CPU, Backend::GPU };

    for (Backend backend : backends) {

        if (backend == Backend::GPU && !gpu) continue;

        for (DemosaicMethod method : methods) {

            auto start = std::chrono::steady_clock::now();

```

C++

```

    Image image = process_demosaic(*raw, backend, method, {});
    Image shown = apply_gamma(image, 2.2f, backend);

    auto end = std::chrono::steady_clock::now();
    double ms = std::chrono::duration<double, std::milli>(end - start).count();

    std::printf("%s %-17s %5dx%-5d %8.1f ms\n", backend == Backend::GPU ? "GPU" : "CPU",
               demosaic_method_name(method), shown.width, shown.height, ms);
}

Backend best = gpu ? Backend::GPU : Backend::CPU;
Image result = process_demosaic(*raw, best, DemosaicMethod::Malvar, {});

save_image("malvar.png", apply_gamma(result, 2.2f, best));
std::printf("saved malvar.png\n");

return 0;
}

```

```

> demosaic_benchmark.exe test_2.DNG
7328x5501 device: NVIDIA GeForce RTX 3080 Ti
CPU binning 2x2      3664x2750      254.2 ms
CPU bilinear         7328x5501      1012.8 ms
CPU Malvar-He-Cutler 7328x5501      1038.5 ms
GPU binning 2x2      3664x2750      21.2 ms
GPU bilinear         7328x5501      56.4 ms
GPU Malvar-He-Cutler 7328x5501      60.3 ms
saved malvar.png

```

ТЕКСТ

Первый вызов на GPU перед циклом нужен для «прогрева»: при первом обращении CUDA создаёт контекст, и это заняло бы лишние сотни миллисекунд в замеры.

Типы данных

RawImage

Мозаика сенсора и всё, что про неё известно. Объявлен в `im_read.hpp`.

ПОЛЕ	ТИП	ЧТО ХРАНИТ
<code>width</code> , <code>height</code>	<code>int</code>	размер активной области
<code>black_level</code> , <code>white_level</code>	<code>int</code>	уровень чёрного и насыщения, ADU
<code>cfaPattern</code>	<code>unsigned</code>	узор Байера в формате LibRaw <code>filters</code>
<code>wbR</code> , <code>wbG</code> , <code>wbB</code>	<code>float</code>	баланс белого камеры, делённый на зелёный (<code>wbG = 1</code>)
<code>data</code>	<code>std::vector<uint16_t></code>	пиксели построчно, <code>width × height</code> значений
<code>flip</code>	<code>int</code>	поворот, записанный камерой
<code>activeX</code> , <code>activeY</code>	<code>int</code>	смещение активной области в полном кадре сенсора

ПОЛЕ	ТИП	ЧТО ХРАНИТ
<code>fullWidth</code> , <code>fullHeight</code>	<code>int</code>	полный размер сенсора
<code>obData</code> , <code>obWidth</code> , <code>obHeight</code>	<code>std::vector<uint16_t></code> , <code>int</code>	зона optical black, если найдена
<code>camera</code> , <code>lens</code>	<code>std::string</code>	камера и объектив
<code>iso</code> , <code>shutter</code> , <code>aperture</code> , <code>focal</code>	<code>float</code>	ISO, выдержка в секундах, диафрагма, фокусное в мм

МЕТОД	ЧТО ВОЗВРАЩАЕТ
<code>int color_at(int x, int y) const</code>	цвет фильтра пикселя: <code>0</code> R, <code>1</code> G, <code>2</code> B, <code>3</code> второй G
<code>std::size_t size() const</code>	размер <code>data</code> в байтах: <code>width × height × 2</code>

```
std::optional<RawImage> raw = load_raw("test_2.DNG");
int color = raw->color_at(0, 0);
uint16_t value = raw->data[static_cast<std::size_t>(100) * raw->width + 200];
```

C++

Image и Image16

Цветная картинка: `width`, `height`, `channels` и `data` построчно (`uint8_t` у `Image`, `uint16_t` у `Image16`). Каналы идут подряд: R, G, B, R, G, B...

МЕТОД	IMAGE	ЧТО ДЕЛАЕТ
<code>std::size_t byte_size() const</code>		размер <code>data</code> в байтах
<code>bool data_is_empty() const</code>		<code>true</code> , если картинка пустая — например, после отмены
<code>uint8_t& at(int x, int y, int z)</code>		ссылка на канал <code>z</code> пикселя <code>(x, y)</code> с проверкой границ

```
Image image = demosaic_cpu(*raw);
uint8_t red = image.at(10, 20, 0);
image.at(10, 20, 1) = 255;
```

C++

ImageStats и ChannelStats

`ImageStats` — `min`, `max` (`uint16_t`), `mean`, `stdDev`, `rangeBits`, `usefulBits`, `snrDb` (`double`).

ПОЛЕ	ФОРМУЛА
<code>rangeBits</code>	$\log_2(\text{max} - \text{min} + 1)$ — разброс значений в битах
<code>usefulBits</code>	$\log_2((\text{white_level} - \text{black_level}) / \text{stdDev})$ — полезные биты, сколько уровней различимо над шумом
<code>snrDb</code>	$20 \cdot \log_{10}((\text{mean} - \text{black_level}) / \text{stdDev})$

Эти три поля заполняет `finish_bits(ImageStats&, const RawImage&)`; её вызывают `compute_stats`, `compute_region` и `compute_region_channels` после того, как посчитана σ .

`ChannelStats` — `stats[4]` (`ImageStats` для каждой позиции CFA), `counts[4]` (число пикселей), `labels` ("R", "G1", "G2", "B" в порядке позиций).

NoiseReport

Результат двухкадрового метода: массивы из 4 значений по каналам — `diffMean` (среднее разности кадров), `temporal` (временной шум), `total` (полный шум первого кадра), `fpn` (фиксированный узор); `labels`; `valid` — `false`, если кадры разного размера.

DenoiseParams

ПОЛЕ	ПО УМОЛЧАНИЮ	СМЫСЛ
<code>method</code>	<code>DenoiseMethod::Gaussian</code>	<code>Gaussian</code> , <code>Median</code> или <code>Bilateral</code>
<code>sigma</code>	<code>1.0</code>	Spatial σ в пикселях плоскости канала (<code>Gaussian</code> , <code>Bilateral</code>)
<code>medianSize</code>	<code>3</code>	окно медианы: 3 или 5 (поддерживается до 7)
<code>rangeSigma</code>	<code>40.0</code>	Range σ в ADU (<code>Bilateral</code>)

CfaPlane и PlaneIndex

`CfaPlane` — одна плоскость канала: `width`, `height`, `std::vector<float> data`.

`PlaneIndex(const CfaPlane& plane, int reach)` заранее считает таблицы индексов с запасом `reach` по краям. После этого `at(x, y)` возвращает индекс в `data`, даже если `x` или `y` вышли за край не дальше `reach`: подставляется ближайший крайний пиксель.

Перечисления

ТИП	ЗНАЧЕНИЯ	ГДЕ ОБЪЯВЛЕН
<code>Backend</code>	<code>CPU</code> , <code>GPU</code>	<code>process.hpp</code>
<code>DemosaicMethod</code>	<code>Binning</code> , <code>Bilinear</code> , <code>Malvar</code>	<code>process.hpp</code>
<code>DenoiseMethod</code>	<code>Gaussian</code> , <code>Median</code> , <code>Bilateral</code>	<code>denoise.hpp</code>

im_read.hpp — загрузка, демозаик, сохранение

load_raw

```
std::optional<RawImage> load_raw(const std::string& path);
```

C++

Открывает RAW через LibRaw и возвращает мозаику активной области сенсора.

- Кадр обрезается до активной области; отступ округляется до чётного, чтобы не сбить узор Байера.
- Заполняются black и white level, узор `cfaPattern`, баланс белого, EXIF и поворот `flip`.
- Если по краю сенсора есть закрытая от света зона, она копируется в `obData`.
- Возвращает `std::nullopt` и пишет причину в `std::cerr`, если файл не открылся, сенсор X-Trans или DNG уже без мозаики.

```
std::optional<RawImage> raw = load_raw("test.dng");
if (!raw) return 1;
std::printf("%dx%d black %d white %d filters 0x%x\n", raw->width, raw->height,
            raw->black_level, raw->white_level, raw->cfaPattern);
```

C++

load_image

```
std::optional<Image> load_image(const std::string& path, int desiredChannels = 0);
```

C++

Загружает обычную картинку (PNG, JPEG, BMP и другие) через `stb_image`. `desiredChannels = 3` принудительно даёт RGB, `0` — столько каналов, сколько в файле.

```
std::optional<Image> picture = load_image("output.png", 3);
if (picture) std::printf("%dx%d, %d channels\n", picture->width, picture->height, picture->channels);
```

C++

save_image

```
bool save_image(const std::string& path, const Image& image);
```

C++

Сохраняет `Image` в PNG. Возвращает `false`, если записать не удалось.

demosaic_cpu

```
Image demosaic_cpu(const RawImage& raw);
```

C++

Binning 2×2 в одном потоке: результат вдвое меньше, 8 бит, без гаммы.

```
Image small = demosaic_cpu(*raw);
save_image("binning.png", apply_gamma(small, 2.2f));
```

C++

demosaic_cpu_progress

```
Image demosaic_cpu_progress(const RawImage& raw, const std::function<bool(int, int)>& report);
```

C++

Тот же Binning 2×2, но во всех потоках. `report(готово, всего)` вызывается раз в 2 мс с числом готовых строк результата; если он вернёт `false`, работа останавливается и возвращается пустой `Image`. Вместо `report` можно передать `{}`.

```
Image image = demosaic_cpu_progress(*raw, [](int done, int total) {
    std::printf("\r%d / %d rows", done, total);
    return true;
});
```

C++

demosaic_full_progress

```
Image demosaic_full_progress(const RawImage& raw, bool malvar, const std::function<bool(int, int)>& report);
```

C++

Демозаик полного размера во всех потоках: `malvar = false` — Bilinear, `true` — Malvar-He-Cutler. Прогресс и отмена — как у `demosaic_cpu_progress`.

```
Image full = demosaic_full_progress(*raw, true, {});
if (!full.data_is_empty()) save_image("malvar.png", apply_gamma(full, 2.2f));
```

C++

demosaic_cpu16

```
Image16 demosaic_cpu16(const RawImage& raw);
```

C++

Binning 2×2 с 16-битным результатом (0...65535), без гаммы. Используется командой **Save linear to 16-bit TIFF**.

save_rgb_tiff16

```
bool save_rgb_tiff16(const std::string& path, const Image16& image);
```

C++

RGB TIFF, 16 бит на канал. `image.channels` должен быть равен 3.

```
save_rgb_tiff16("linear16.tiff", demosaic_cpu16(*raw));
```

C++

save_raw_binary

```
bool save_raw_binary(const std::string& path, const RawImage& raw);
```

C++

Записывает `raw.data` как есть: `uint16`, little-endian, без заголовка. Размер файла — `raw.size()` байт.

save_raw_notes

```
bool save_raw_notes(const std::string& path, const RawImage& raw,
                   const std::string& cfa, const std::string& data_path);
```

C++

Кладёт рядом с сохранённым файлом обычный текстовый блокнот: ширину, высоту, тип данных, размер в байтах, уровни black и white, узор CFA и название камеры. Без этих чисел двоичный файл прочитать правильно нельзя.

Если `data_path` не пустой, первой строкой пишется готовая команда `READBIN(...)` для Mathcad с этим путём; для TIFF её передают пустой, потому что размеры уже лежат в заголовке самого TIFF.

```
save_raw_binary("region.bin", part);
save_raw_notes("region_mathcad.txt", part, cfa_pattern_name(part), "region.bin");
```

C++

save_raw_in_tiff

```
bool save_raw_in_tiff(const std::string& path, const RawImage& raw);
```

C++

Сохраняет мозаику как серый 16-битный TIFF.

apply_gamma

```
Image apply_gamma(const Image& image, float gamma);
```

C++

$out = 255 \cdot (in / 255)^{(1/gamma)}$ для каждого байта в одном потоке. Версия с выбором CPU или GPU — в `process.hpp`.

parallel_workers

```
int parallel_workers(int rows, int minimumRowsPerWorker = 64);
```

C++

Сколько потоков запускать: не больше числа логических ядер и не больше одного потока на `minimumRowsPerWorker` строк, но не меньше одного. На 12-поточном процессоре для 3024 строк получится 12, для 200 строк — 3.

raw_at

```
uint16_t raw_at(const RawImage& raw, int x, int y);
```

C++

Значение пикселя. Координаты за краем кадра прижимаются к ближайшему краю — удобно для окон фильтров.

image_stats.hpp — статистика и шум

compute_region_channels

```
ChannelStats compute_region_channels(const RawImage& raw, int x0, int y0, int width, int height); C++
```

Статистика области отдельно для каждого канала, в несколько потоков. Область автоматически обрезается по кадру.

```
ChannelStats c = compute_region_channels(*raw, 3712, 1152, 256, 256);  
for (int k = 0; k < 4; ++k)  
    std::printf("%s mean %.1f sigma %.2f (%zu px)\n", c.labels[k].c_str(),  
                c.stats[k].mean, c.stats[k].stdDev, c.counts[k]); C++
```

compute_region

```
ImageStats compute_region(const RawImage& raw, int x0, int y0, int width, int height); C++
```

Статистика области по всем пикселям вместе, в несколько потоков. Область должна лежать внутри кадра: эта функция её не обрезает. Если не уверен, сначала вызови `clamp_region`.

```
int x = 3712, y = 1152, w = 256, h = 256;  
clamp_region(*raw, x, y, w, h);  
ImageStats s = compute_region(*raw, x, y, w, h);  
std::printf("mean %.1f sigma %.2f\n", s.mean, s.stdDev); C++
```

compute_stats

```
ImageStats compute_stats(const RawImage& raw); C++
```

Статистика всего кадра по всем пикселям, в одном потоке.

clamp_region

```
void clamp_region(const RawImage& raw, int& x0, int& y0, int& width, int& height); C++
```

Сдвигает и обрезает область так, чтобы она целиком лежала в кадре; минимальный размер 1×1.

region_crop

```
RawImage region_crop(const RawImage& raw, int x0, int y0, int width, int height); C++
```

Вырезает область в отдельный `RawImage`. Начало выравнивается на чётные координаты, размер округляется до чётного, чтобы узор Байера остался прежним. Копируются уровни, узор и баланс белого. Так работает **Demosaic selected region**.

```
RawImage part = region_crop(*raw, 4200, 3700, 320, 200);
Image preview = demosaic_full_progress(part, true, {});
```

C++

cfa_index, cfa_labels, cfa_pattern_name

```
int cfa_index(int x, int y);
std::array<std::string, 4> cfa_labels(const RawImage& raw);
std::string cfa_pattern_name(const RawImage& raw);
```

C++

- `cfa_index` — позиция пикселя в блоке 2×2: $((y \& 1) \ll 1) | (x \& 1)$, то есть 0...3.
- `cfa_labels` — имя канала для каждой позиции: для RGGB {"R", "G1", "G2", "B"}, для BGGR {"B", "G1", "G2", "R"}.
- `cfa_pattern_name` — узор строкой: "RGGB", "BGGR", "GRBG" или "GBRG".

```
std::array<std::string, 4> labels = cfa_labels(*raw);
std::printf("pixel (5, 8) is %s in %s\n", labels[cfa_index(5, 8)].c_str(),
cfa_pattern_name(*raw).c_str());
```

C++

compute_histogram

```
std::array<std::vector<uint32_t>, 4> compute_histogram(const RawImage& raw, int x0, int y0,
int width, int height, int bins);
```

C++

Гистограмма значений по каналам: `bins` столбцов на диапазон 0...white level. Номер столбца — $v / \text{white_level} \cdot (\text{bins} - 1)$.

```
auto hist = compute_histogram(*raw, 0, 0, raw->width, raw->height, 256);
auto labels = cfa_labels(*raw);
for (int k = 0; k < 4; ++k) {
    auto peak = std::max_element(hist[k].begin(), hist[k].end()) - hist[k].begin();
    std::printf("%s: most pixels in bin %d\n", labels[k].c_str(), static_cast<int>(peak));
}
```

C++

compute_noise_histogram

```
std::array<std::vector<uint32_t>, 4> compute_noise_histogram(const RawImage& raw, int x0, int y0,
int width, int height, int bins, double span, const std::array<double, 4>& mean);
```

C++

Гистограмма отклонений $v - \text{mean}[k]$ в диапазоне $\pm \text{span}$; отклонения дальше пропускаются. Если $2 \cdot \text{span} / (\text{bins} - 1)$ — целое число, каждый столбец покрывает ровно целое число ADU.

```
ChannelStats c = compute_region_channels(*raw, x, y, 256, 256);
std::array<double, 4> mean = { c.stats[0].mean, c.stats[1].mean, c.stats[2].mean, c.stats[3].mean };
auto noise = compute_noise_histogram(*raw, x, y, 256, 256, 161, 80.0, mean);
```

161 столбец на ± 80 ADU — ровно 1 ADU на столбец.

compute_profile

```
std::vector<double> compute_profile(const RawImage& raw, int x0, int y0, int width, int height, bool alongX);
```

`alongX = true` — среднее по каждому столбцу области (длина результата `width`), `false` — по каждой строке (длина `height`).

```
std::vector<double> row = compute_profile(*raw, 0, 3800, raw->width, 1, true);
```

compute_two_frame

```
NoiseReport compute_two_frame(const RawImage& first, const RawImage& second,
                              int x0, int y0, int width, int height);
```

Двухкадровый шум по каналам (формулы — в разделе [Шум](#)). `valid = false`, если кадры пустые или разного размера.

```
std::optional<RawImage> first = load_raw("flat_1.dng");
std::optional<RawImage> second = load_raw("flat_2.dng");
NoiseReport report = compute_two_frame(*first, *second, 1000, 1000, 256, 256);
if (report.valid)
    for (int k = 0; k < 4; ++k)
        std::printf("%s rd=%.2f fpn=%.2f tot=%.2f\n", report.labels[k].c_str(),
                    report.temporal[k], report.fpn[k], report.total[k]);
```

subtract_dark

```
RawImage subtract_dark(const RawImage& light, const RawImage& dark);
```

Возвращает `light - dark + black_level` с обрезкой в 0...white level. Если размеры кадров разные, возвращает `light` без изменений.

```
std::optional<RawImage> dark = load_raw("dark.dng");
RawImage clean = subtract_dark(*raw, *dark);
```

denoise.hpp — шумоподавление

denoise_frame

```
bool denoise_frame(RawImage& raw, int x0, int y0, int width, int height,
                  const DenoiseParams& params, const std::function<bool(int, int)>& report);
```

C++

Главная функция: фильтрует прямоугольник кадра **на месте**, отдельно для каждого канала CFA (схема — в разделе [Шум](#)).

- Возвращает `true`, если фильтр закончил, и `false`, если `report` вернул `false`. После отмены кадр не меняется.
- `report(готово, всего)` вызывается раз в 2 мс; единица работы — одна строка плоскости за один проход.

```
DenoiseParams params;
params.method = DenoiseMethod::Bilateral;
params.sigma = 1.0;
params.rangeSigma = 50.0;

std::atomic<bool> cancel{false};

RawImage frame = *raw;
bool finished = denoise_frame(frame, 0, 0, frame.width, frame.height, params,
    [&](int done, int total) {
        std::printf("\r%d%", done * 100 / std::max(1, total));
        return !cancel.load();
    });
```

C++

denoise_gaussian_plane, denoise_median_plane, denoise_bilateral_plane

```
bool denoise_gaussian_plane(const CfaPlane& source, CfaPlane& result, double sigma,
                           std::atomic<int>& progress, int total,
                           std::atomic<bool>& stopped, const std::function<bool(int, int)>& report);

bool denoise_median_plane(const CfaPlane& source, CfaPlane& result, int size,
                          std::atomic<int>& progress, int total,
                          std::atomic<bool>& stopped, const std::function<bool(int, int)>& report);

bool denoise_bilateral_plane(const CfaPlane& source, CfaPlane& result, double sigma, double
rangeSigma,
                           std::atomic<int>& progress, int total,
                           std::atomic<bool>& stopped, const std::function<bool(int, int)>&
report);
```

C++

Фильтруют одну плоскость канала из `source` в `result`. Обычно их вызывает `denoise_frame`, но можно и напрямую. Параметры `progress`, `total` и `stopped` общие для всех четырёх плоскостей, чтобы прогресс шёл от 0 до 100 % через весь кадр.

```
CfaPlane plane;
plane.width = 64;
```

C++

```
plane.height = 64;
plane.data.assign(64 * 64, 100.f);

CfaPlane smooth;
std::atomic<int> progress{0};
std::atomic<bool> stopped{false};

denoise_gaussian_plane(plane, smooth, 1.0, progress, 64 * 2, stopped, {});
```

У Gaussian два прохода (по строкам и по столбцам), поэтому `total` для плоскости 64×64 равен 128.

denoise_run_rows

```
bool denoise_run_rows(int rows, const std::function<void(int)>& body,
    std::atomic<int>& progress, int total,
    std::atomic<bool>& stopped, const std::function<bool(int, int)>& report);
```

Выполняет `body(y)` для каждой строки `0...rows-1` в нескольких потоках (`parallel_workers(rows, 16)`), обновляет `progress` и раз в 2 мс опрашивает `report`. Возвращает `false`, если работу остановили.

denoise_radius, denoise_passes, denoise_method_name

```
int denoise_radius(const DenoiseParams& params);
int denoise_passes(const DenoiseParams& params);
const char* denoise_method_name(DenoiseMethod method);
```

- `denoise_radius` — радиус окна: `[30]` для Gaussian, `[20]` для Bilateral, половина окна для Median.
- `denoise_passes` — число проходов на плоскость: 2 для Gaussian, 1 для остальных.
- `denoise_method_name` — `"gaussian"`, `"median"` или `"bilateral"`.

process.hpp — выбор устройства и метода

process_demosaic

```
Image process_demosaic(const RawImage& raw, Backend backend);
Image process_demosaic(const RawImage& raw, Backend backend, const std::function<bool(int, int)>& report);
Image process_demosaic(const RawImage& raw, Backend backend, DemosaicMethod method,
    const std::function<bool(int, int)>& report);
```

Одна точка входа для демозаика. Первые две версии всегда делают Binning 2×2, третья вызывает нужную реализацию:

МЕТОД	CPU	GPU
Binning	<code>demosaic_cpu_progress</code>	<code>process_gpu_demosaic</code>
Bilinear	<code>demosaic_full_progress(raw, false, ...)</code>	<code>process_gpu_demosaic_full(raw, false, ...)</code>
Malvar	<code>demosaic_full_progress(raw, true, ...)</code>	<code>process_gpu_demosaic_full(raw, true, ...)</code>

Функции GPU бросают `std::runtime_error` при ошибке CUDA, поэтому вызов лучше обернуть:

```
Backend backend = cuda_device_available() ? Backend::GPU : Backend::CPU;

try {
    Image image = process_demosaic(*raw, backend, DemosaicMethod::Malvar, {});
    save_image("result.png", apply_gamma(image, 2.2f, backend));
} catch (const std::exception& error) {
    std::printf("demosaic failed: %s\n", error.what());
}
```

apply_gamma (с выбором устройства)

```
Image apply_gamma(const Image& image, float gamma, Backend backend);
```

Гамма на CPU (`apply_gamma(image, gamma)`) или на GPU (`gammaApply_GPU`).

demosaic_method_name

```
const char* demosaic_method_name(DemosaicMethod method);
```

"binning 2x2", "bilinear" или "Malvar-He-Cutler".

gpu_process.hpp — CUDA

cuda_device_available, cuda_device_name, cuda_memory_usage

```
bool cuda_device_available();
std::string cuda_device_name();
bool cuda_memory_usage(std::size_t& usedMb, std::size_t& totalMb);
```

- `cuda_device_available` — есть ли хотя бы одно CUDA-устройство. Без драйвера NVIDIA или со слишком старым драйвером возвращает `false`, а не падает.
- `cuda_device_name` — название видеокарты или "no CUDA device".
- `cuda_memory_usage` — занятая и полная видеопамять в МБ.

```
if (cuda_device_available()) {
    std::size_t used = 0, total = 0;
    if (cuda_memory_usage(used, total))
```

```
std::printf("%s: %zu / %zu MB\n", cuda_device_name().c_str(), used, total);
}
```

cudaCheck

```
void cudaCheck(cudaError_t status, const char* what);
```

C++

Если `status` не `cudaSuccess`, бросает `std::runtime_error` с текстом `what` и описанием ошибки CUDA.

process_gpu_demosaic, process_gpu_demosaic_full

```
Image process_gpu_demosaic(const RawImage& raw);
Image process_gpu_demosaic(const RawImage& raw, const std::function<bool(int, int)>& report);
Image process_gpu_demosaic_full(const RawImage& raw, bool malvar, const std::function<bool(int, int)>& report);
```

C++

Демозаик на видеокарте:

1. выделить видеопамять;
2. загрузить мозаику;
3. запустить ядро — полосами по 256 строк результата для Binning и по 512 строк для полного размера, с вызовом `report` после каждой полосы;
4. скачать результат и освободить память.

При отмене возвращают пустой `Image`, при ошибке CUDA бросают исключение.

gammaApply_GPU

```
Image gammaApply_GPU(const Image& image, float gamma);
```

C++

Гамма на видеокарте, результат совпадает с `apply_gamma(image, gamma)`.

im_process.cuh — ядра CUDA

Функции запуска вызываются с **указателями на видеопамять** и работают **асинхронно**: после вызова нужно дождаться `cudaDeviceSynchronize()`.

```
void launchDemosaicBin(const uint16_t* d_raw, int rawW, int rawH, uint8_t* d_out, int outW, int outH,
                      int blackLevel, int whiteLevel, unsigned cfaPattern,
                      float wbR, float wbG, float wbB, cudaStream_t stream = 0);

void launchDemosaicFull(const uint16_t* d_raw, int rawW, int rawH, uint8_t* d_out, int yStart, int
rows,
                      int blackLevel, int whiteLevel, unsigned cfaPattern,
                      float wbR, float wbG, float wbB, int malvar, cudaStream_t stream = 0);
```

```
void launchApplyGamma(const uint8_t* d_src, uint8_t* d_dst, size_t count, float invGamma,
                      cudaStream_t stream = 0);
```

ФУНКЦИЯ	ЯДРО	ЧТО ДЕЛАЕТ
<code>launchDemosai cBin</code>	<code>demosaicBin Kernel</code>	Binning 2×2, блоки потоков 16×16
<code>launchDemosai cFull</code>	<code>demosaicFul lKernel</code>	Bilinear (<code>malvar = 0</code>) или Malvar (<code>malvar = 1</code>) для строк <code>yStart ... yStart + rows - 1</code> ; соседей читает <code>sampleRow</code> с прижатием к краю
<code>launchApplyGa mma</code>	<code>applyGammaK ernel</code>	$255 \cdot (v/255)^{\text{invGamma}}$ для <code>count</code> байтов, по 256 потоков в блоке

Так их использует `process_gpu_demosaic`:

```
int outW = raw.width / 2;
int outH = raw.height / 2;
size_t outBytes = static_cast<size_t>(outW) * outH * 3;

uint16_t* d_raw = nullptr;
uint8_t* d_out = nullptr;

cudaCheck(cudaMalloc(reinterpret_cast<void*>(&d_raw), raw.size()), "cudaMalloc raw");
cudaCheck(cudaMalloc(reinterpret_cast<void*>(&d_out), outBytes), "cudaMalloc out");
cudaCheck(cudaMemcpy(d_raw, raw.data.data(), raw.size(), cudaMemcpyHostToDevice), "upload");

launchDemosaicBin(d_raw, raw.width, raw.height, d_out, outW, outH,
                  raw.black_level, raw.white_level, raw.cfaPattern, raw.wbR, raw.wbG, raw.wbB);
cudaCheck(cudaDeviceSynchronize(), "demosaic kernel");

Image image;
image.width = outW;
image.height = outH;
image.channels = 3;
image.data.resize(outBytes);

cudaCheck(cudaMemcpy(image.data.data(), d_out, outBytes, cudaMemcpyDeviceToHost), "download");

cudaFree(d_raw);
cudaFree(d_out);
```

main.cpp — консольная проверка

```
cuda_raw_processor.exe [файл.dng]
```

ТЕКСТ

Без аргумента открывает `test.dng` в текущей папке. Печатает размер, black и white level и узор, затем сохраняет `raw_data.bin`, `raw_mosaic.tiff`, `output_cpu.png` (Binning на CPU) и `output_gpu.png` (Binning на GPU).

ЗАМЕТКА

Консольная программа не проверяет наличие видеокарты: на компьютере без NVIDIA вызов `process_gpu_demoaic` завершится исключением. Приложение с окном проверяет устройство и переходит на CPU.

gui.cpp — приложение

Классы

КЛАСС	ЧТО ДЕЛАЕТ
<code>MainWindow</code>	главное окно: собирает интерфейс и связывает кнопки с обработкой
<code>TitleBar</code>	собственный заголовок окна с кнопками; перетаскивание и изменение размера — через <code>nativeEvent</code>
<code>BackgroundWidget</code>	фон центральной области
<code>ImageLabel</code>	область просмотра: зум, панорама, выделение с маркерами, лупа, миникарта, линейки, сетка, сравнение, перетаскивание файлов
<code>HistogramWidget</code>	гистограмма четырёх каналов
<code>ProfileWidget</code>	график профиля
<code>NoiseCompareWidget</code>	график «до / после» с осями, $\pm\sigma$ и логарифмической шкалой
<code>NoisePage</code>	вкладка NOISE · FILTERS: кнопки каналов, карточки, таблица, панель фильтра
<code>NoiseSnapshot</code>	снимок статистики и гистограмм области для «до» и «после»
<code>Document</code>	состояние одного открытого файла: кадр, оригинал, области, гамма, зум, поворот
<code>RegionsCommand</code> , <code>FrameCommand</code>	шаги истории отмены для списка областей и для кадра

Методы MainWindow

ГРУППА	МЕТОДЫ	ЧТО ДЕЛАЮТ
Файлы	<code>onLoad</code> , <code>loadRawFile</code> , <code>onLoadReference</code>	диалог выбора, загрузка файла в новую вкладку, загрузка reference-кадра
Вкладки файлов	<code>stashDocument</code> , <code>activateDocument</code> , <code>closeDocument</code> , <code>resetToEmpty</code>	сохранить состояние текущего файла, переключиться, закрыть, очистить окно
Фоновые задачи	<code>startTask</code> , <code>finishTask</code>	запустить работу в отдельном потоке с прогрессом и отменой; разблокировать кнопки и показать результат
Демозаик	<code>onProcess</code> , <code>onProcessRegion</code> , <code>setDemoaicMethod</code> , <code>onGammaChanged</code> , <code>applyPendingGamma</code>	запуск демозаика для кадра или области; выбор метода; гамма с задержкой 50 мс, чтобы ползунок не тормозил

ГРУППА	МЕТОДЫ	ЧТО ДЕЛАЮТ
Области	<code>onRegionSelected</code> , <code>refreshAnalysis</code> , <code>applyNumericRegion</code> , <code>onSelectedRow</code> , <code>onSelectWholeColumn</code> , <code>nudgeRegion</code>	выделить область и пересчитать статистику, гистограмму и профиль
Список областей	<code>addRegion</code> , <code>removeRegion</code> , <code>clearRegions</code> , <code>rebuildRegionTable</code> , <code>applyRegionList</code>	изменить список через историю отмены и перерисовать таблицу
Шум	<code>compareFrames</code> , <code>subtractReference</code> , <code>measureOpticalBlack</code> , <code>applyMeasuredBlack</code> , <code>revertOriginal</code>	двухкадровый шум, темновой кадр, optical black, возврат к оригиналу
Вкладка шума	<code>showPage</code> , <code>refreshNoisePage</code> , <code>redrawNoisePage</code> , <code>makeNoiseSnapshot</code> , <code>fitHistogramRange</code> , <code>noiseChannel</code>	переключение вкладок; пересчёт снимков «до/после»; подбор диапазона гистограммы с шагом в целое число ADU
Фильтры	<code>applyDenoise</code> , <code>denoiseLabel</code> , <code>autoRangeSigma</code> , <code>logNoiseChange</code>	запуск фильтра в фоне и запись результата в историю; Auto для Range σ ; запись изменения σ в журнал
Эталон «до»	<code>captureNoiseBefore</code> , <code>autoCaptureBefore</code> , <code>useOriginalAsBefore</code> , <code>resetNoiseReference</code>	запомнить «до» вручную, автоматически перед изменением, взять оригинал, сбросить
Кадр и история	<code>applyRawFrame</code>	подставить кадр из истории отмены и обновить всё, что от него зависит
Экспорт	<code>saveBin</code> , <code>saveRegionBin</code> , <code>saveTiff</code> , <code>saveRegionTiff</code> , <code>savePng</code> , <code>saveLinearTiff16</code> , <code>exportStats</code> , <code>exportRegions</code> , <code>writeCsvMeta</code> , <code>savePreset</code> , <code>loadPreset</code>	сохранение файлов, CSV и пресетов; <code>exportFolder</code> создаёт папку по имени файла и возвращает путь внутри неё, <code>writeExportNotes</code> кладёт туда же текстовый блокнот с размерами и уровнями
Вид	<code>rebuildPreview</code> , <code>rawToQt</code> , <code>imageToQt</code> , <code>toggleCompare</code> , <code>setRotation</code> , <code>setZoom</code> , <code>toggleFullScreen</code> , <code>togglePanels</code> , <code>copyViewToClipboard</code>	собрать картинку для экрана (с клиппингом и поворотом) и управлять видом
Состояние	<code>updateHud</code> , <code>updateCursorReadout</code> , <code>updateStatusFile</code> , <code>updateMetaInfo</code> , <code>updateMemoryUsage</code> , <code>logLine</code>	HUD, строка состояния, FILE INFO, память, журнал
Настройки	<code>loadSettings</code> , <code>saveSettings</code>	восстановить и сохранить настройки и расположение панелей

Сборка и GitHub

Как собрать программу из исходников, сделать установщик, отправить проект на GitHub и поставить программу на другой компьютер.

Что установить для сборки

Для того чтобы **пользоваться** программой, всё это не нужно — достаточно установщика. Инструменты нужны только тому, кто собирает программу из исходников.

ИНСТРУМЕНТ	ВЕРСИЯ В ПРОЕКТЕ	ЗАЧЕМ
Visual Studio 2022 Build Tools, нагрузка «Разработка классических приложений на C++»	MSVC 19.39	компилятор и линковщик
CMake	4.4 (подойдёт 3.18 и новее)	описание сборки
Qt	6.7.3, MSVC 2019 64-bit, в <code>C:\Qt\6.7.3\msvc2019_64</code>	окно и интерфейс
NVIDIA CUDA Toolkit	13.0	GPU-код; нужен для сборки даже без видеокарты
vcpkg	в <code>C:\vcpkg</code>	библиотеки LibRaw и libtiff
Git	любая свежая	репозиторий
Inno Setup	6.7.3	установщик

Библиотеки через vcpkg:

```
git clone https://github.com/microsoft/vcpkg C:\vcpkg
C:\vcpkg\bootstrap-vcpkg.bat
C:\vcpkg\vcpkg install libraw:x64-windows tiff:x64-windows
```

POWERSHELL

ЗАМЕТКА

`CMakeLists.txt` ищет Qt в `C:/Qt/6.7.3/msvc2019_64`. Если Qt стоит в другом месте, добавь к настройке CMake параметр `-DCMAKE_PREFIX_PATH=путь\к\Qt`.

Сборка для разработки (Debug)

Все команды — в консоли **x64 Native Tools Command Prompt for VS 2022** из папки проекта. Полный путь к `cmake.exe` нужен, если в `PATH` раньше стоит CMake из MSYS2.

```
"C:\Program Files\CMake\bin\cmake.exe" -S . -B build -G Ninja ^
-DCMAKE_BUILD_TYPE=Debug ^
```

CMD

```
-DCMAKE_TOOLCHAIN_FILE=C:/vcpkg/scripts/buildsystems/vcpkg.cmake
```

```
"C:\Program Files\CMake\bin\cmake.exe" --build build --target cuda_raw_gui
```

При первой сборке рядом с exe нужно положить отладочные DLL Qt:

```
C:\Qt\6.7.3\msvc2019_64\bin\windeployqt.exe --debug build\cuda_raw_gui.exe  
build\cuda_raw_gui.exe
```

CMD

Если программа запущена, новая сборка упадёт с ошибкой `LNK1168`: закрой программу и собери снова.

Release, папка программы и установщик

Одна команда делает всё:

```
powershell -ExecutionPolicy Bypass -File installer\build_installer.ps1
```

POWERSHELL

Что делает скрипт:

1. собирает Release в `build-release` под видеокарты NVIDIA от RTX 20xx до RTX 50xx;
2. копирует `cuda_raw_gui.exe` и DLL из vcpkg в `dist\RawImageProcess`;
3. запускает `windeployqt`, который добавляет DLL и плагины Qt;
4. добавляет библиотеки Visual C++ и CUDA runtime;
5. упаковывает папку в `dist\RawImageProcess-1.0-portable.zip`;
6. если найден Inno Setup, собирает `dist\RawImageProcess-1.0-Setup.exe` по сценарию `installer\RawImageProcess.iss`.

Параметр `-SkipBuild` пропускает компиляцию и только собирает папку и установщик заново. Версия берётся из `kAppVersion` в `gui.cpp`.

Чем Release отличается от Debug

В Release включена оптимизация компилятора и выключены отладочные проверки, поэтому фильтры шума работают в 3.4–6.2 раза быстрее. Ещё Release собирается как обычное оконное приложение, без чёрной консоли за окном. Для отладки используйте Debug, людям отдавай Release.

Проверка на «чистой» системе без видеокарты

У разработчика в `PATH` прописаны Qt и vcpkg, поэтому программа может случайно найти DLL, которых нет в папке. Проверить это можно так:

```
set PATH=C:\Windows\System32;C:\Windows  
set CUDA_VISIBLE_DEVICES=-1  
dist\RawImageProcess\cuda_raw_gui.exe
```

CMD

Первая строка оставляет только системные папки Windows, вторая прячет видеокарту от CUDA. Должно открыться окно с серой кнопкой GPU и надписью `no CUDA device` в строке состояния. После проверки закрой эту консоль.

Ещё надёжнее — **Песочница Windows** (Windows Sandbox) в Windows 10/11 Pro: чистая временная Windows без Qt, vsrkg и CUDA. Включается в «Включение или отключение компонентов Windows», после перезагрузки в неё можно скопировать Setup.exe и установить.

GitHub

Репозиторий проекта: `https://github.com/SolomonDi/Image_Process`.

Что сейчас лежит в репозитории

Вместе с исходниками в репозиторий когда-то попали файлы, которым там не место:

ФАЙЛ	РАЗМЕР	ЧТО С НИМ СДЕЛАТЬ
<code>raw_data.bin</code>	80 МБ	результат работы программы; убрать из репозитория. GitHub предупреждает о файлах больше 50 МБ и не принимает файлы больше 100 МБ
<code>im_process.obj</code>	57 КБ	промежуточный файл компиляции; убрать
<code>aqtinstall.log</code>	26 КБ	журнал установки Qt; убрать

В `.gitignore` уже добавлены `*.bin`, `*.log`, `build-release/`, `dist/` и `practice/`, чтобы такие файлы больше не попадали в коммиты. Новые файлы, которые нужно добавить: `denoise.hpp`, `installer/`, `docs/`, `README.md`.

Отправить изменения

В PowerShell или обычной консоли из папки проекта:

```
cd "C:\Users\RED.DOT\Desktop\PNG NOISE"
git status
git rm --cached raw_data.bin im_process.obj aqtinstall.log
git add .gitignore CMakeLists.txt README.md denoise.hpp gui.cpp gpu_process.hpp im_process.cpp
im_process.cuh im_read.hpp image_stats.hpp process.hpp installer docs
git status
git commit -m "Feat: Dmitry Solomonov. Noise page, denoise filters, installer and docs"
git push origin main
```

POWERSHELL

КОМАНДА	ЗАЧЕМ
<code>git status</code>	посмотреть, что изменено; перед коммитом — проверить, что в списке нет лишнего
<code>git rm --cached ...</code>	убрать файлы из репозитория; на диске они остаются
<code>git add ...</code>	отметить файлы для коммита
<code>git commit -m "..."</code>	сохранить снимок изменений с описанием

КОМАНДА	ЗАЧЕМ
<code>git push origin main</code>	отправить коммит на GitHub

ЗАМЕТКА

`git rm --cached` убирает `raw_data.bin` из новых коммитов, но файл остаётся в истории. При клонировании он скачается один раз вместе с историей. Если репозиторий нужно сделать маленьким, историю можно почистить утилитой `git filter-repo`, но это отдельная аккуратная операция.

Если `git push` пишет `rejected ... fetch first`, значит, на GitHub есть коммиты, которых нет у тебя. Сначала подтяни их, потом отправь:

```
git pull --rebase origin main
git push origin main
```

POWERSHELL

Выложить установщик в Releases

Файлы `.exe` не хранят в самом репозитории (они в `.gitignore`), для готовых программ у GitHub есть страница **Releases**.

1. Собери установщик: `powershell -ExecutionPolicy Bypass -File installer\build_installer.ps1`.
2. Открой `https://github.com/SolomonDi/Image_Process` и справа нажми **Releases** → **Draft a new release**.
3. **Choose a tag** → впиши `v1.0` → **Create new tag**.
4. **Release title:** `Raw Image Process 1.0`; в описании кратко перечисли, что нового.
5. Перетащи в поле файлов `dist\RawImageProcess-1.0-Setup.exe` и `dist\RawImageProcess-1.0-portable.zip`.
6. Нажми **Publish release**.

То же самое из консоли, если установить GitHub CLI (`winget install --id GitHub.cli -e`) и войти (`gh auth login`):

```
gh release create v1.0 dist\RawImageProcess-1.0-Setup.exe dist\RawImageProcess-1.0-portable.zip --
title "Raw Image Process 1.0"
```

POWERSHELL

Установка на другой компьютер

Чтобы просто пользоваться

1. Открой `https://github.com/SolomonDi/Image_Process/releases`.

2. Скачай `RawImageProcess-1.0-Setup.exe` и установи. Если Windows покажет предупреждение SmartScreen — **Подробнее → Выполнить в любом случае**.
3. Без установки: скачай `RawImageProcess-1.0-portable.zip`, распакуй и запусти `cuda_raw_gui.exe`.

Нужна Windows 10 версии 1809 или новее либо Windows 11, 64-бит. Видеокарта не обязательна.

Чтобы продолжать разработку

1. Установи инструменты из таблицы в начале раздела.
2. Склонируй репозиторий и собери:

```
git clone https://github.com/SolomonDi/Image_Process.git
cd Image_Process
powershell -ExecutionPolicy Bypass -File installer\build_installer.ps1
```

POWERSHELL

1. Тестовые RAW-файлы (`test.dng`, `test_2.DNG`) в репозитории не хранятся — они большие. Перенеси их отдельно: флешкой или через облако.

Частые проблемы

СООБЩЕНИЕ	ПРИЧИНА И РЕШЕНИЕ
Could not find a package configuration file provided by "Qt6"	Qt не в <code>C:\Qt\6.7.3\msvc2019_64</code> ; укажи <code>-DCMAKE_PREFIX_PATH</code>
No CMAKE_CUDA_COMPILER could be found	не установлен CUDA Toolkit
Could not find a package configuration file provided by "libraw"	не выполнен <code>vcpkg install libraw</code> или не указан <code>CMAKE_TOOLCHAIN_FILE</code>
LNK1168: cannot open cuda_raw_gui.exe for writing	программа запущена; закрой её
could not find or load the Qt platform plugin "windows"	рядом с exe нет папки <code>platforms</code> ; запусти <code>windeployqt</code>
MSVCP140.dll или VCRUNTIME140_1.dll не найден	не скопированы библиотеки Visual C++; пересобери скрипт или установи Visual C++ Redistributable
File raw_data.bin is 80.00 MB; this is larger than GitHub's recommended maximum	выполни <code>git rm --cached raw_data.bin</code>
«Windows защитила ваш компьютер»	у установщика нет цифровой подписи; «Подробнее → Выполнить в любом случае»

Собрано из README.md и docs/*.md репозитория SolomonDi/Image_Process · 13.09.2026